TM Forum Introductory Guide

AI Agent MAS Intent-Based Ontology Operation Service Management Model

IG1421

Maturity Level: Alpha	Team Approved Date: 10-Sep-2025
Release Status: Pre-production	Approval Status: Member Evaluated
Version 1.1.0	IPR Mode: RAND

©TM Forum 2025. All Rights Reserved.

Notice

Copyright © TM Forum 2025. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the TM FORUM IPR Policy, must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an "AS IS" basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Direct inquiries to the TM Forum office:

100 Enterprise Drive, Suite 301 #1649
Rockaway, NJ 07866, USA
Tel No. +1 862 227 1648
TM Forum Web Page: www.tmforum.org

Table of Contents

Notice 2
Table of Contents 3
List of Tables 6
List of Figures 8
Executive Summary 9
Introduction 11
1. Scope and Objectives 13
2. AI Agent MAS Taxonomy of AN Service Management 14
2.1. AI Agent Functional Role 14
2.2. AI Agent Decision-Making 17
2.3. AI Agent Interaction Model 20
2.4. AI Agent Deployment Scope 20
3. AI Agent MAS Intent-Based Interaction Model 23
3.1. Autonomous AI Agents 23
3.2. Collaborative AI Agents 24
3.3. Hierarchical AI Agents 24
3.4. Centralized Coordination Agents 25
3.5. Distributed Coordination Agents 26
3.6. Market-Based Coordination AI Agents 27
3.7. Contract-Based Coordination AI Agents 27
3.8. Coalition-Based Coordination AI Agents 28
3.9. AI Agent Simulation & Inference as a Coordination Mechanism . 30
3.10. Self-Healing in Intent-Based AI Agent MAS Service Management 36
4. AI Agent MAS Intent-Based Service Management 38
4.1. AI Agent MAS Intent-Based Functional Entities 38
4.2. AI Agent MAS Intent-Based Service Management AD 42
4.3. Deployment Scenario 43
5. AI Agent MAS and IMF 54

© TM Forum 2025. All Rights Reserved.

5.1. AI Agent and MAS Intent-Base Framework for AN Service Management 54
5.2. Knowledge Graph and Knowledge Base Representation of Intent Management State 56
5.3. AI Agent Roles in Generating and Updating Knowledge Graph 57
5.4. Semantic Nature of Intent Expressions and Their Operational Realization 59
5.5. Key Roles in the Intent Pipeline 59
6. Intent-Based Service Management TMF APIs Ontology Support 61
6.1. AI Agents MAS Intent based Ontology Support APIs 62
7. AI Agent Intent-Based Ontology for Autonomous Domain Model 72
7.1. Ontology Representation Components 72
7.2. AI Agent Intent-Based Flows 72
8. Appendix A: IMF for AD - AI Agent Protégé Ontology 75
8.2. Turtle File for MAS AI Agent Protégé Ontology 79
9. Appendix B: MAS AI Agent Intent-Based AN Operations – Formulation and Analytics 84
9.1. AI-Agent MAS AN 84
9.2. AI Agent Enabled Intent Management 85
9.3. AD Hierarchy Model 85
9.4. Service Utility Function 88
9.5. Cross-Domain Coordination and Negotiation Model 90
9.6. Intent Management Function (IMF) Model 91
10. Appendix C: Simulation vs. Emulation vs. Digital Twin – Tradeoffs in Resource-Level Coordination 98
10.1. Hybrid Telemetry / Partial-Twin Flow 99
11. Appendix D: Integration of AI Agent MAS Framework 102
12. Appendix E: Carrier-Influenced BGP Path Selection 104
13. Appendix F: Best Practices 105
14. Appendix G: T-Box and A-Box 106
14.1. T-Box (Terminological Box) 106
14.2. A-Box (Assertional Box) 106

© TM Forum 2025. All Rights Reserved.

15. Appendix H: Standards-Aligned Intent Execution Template 108
15.1. Intent Model RFC 9315 and RFC9316 108
15.2. TMF Intent Model Artifacts and Alignment 111
16. Terms & Abbreviations Used within this Document 112
16.1. Terminology 112
16.2. Abbreviations & Acronyms 112
17. References 113
18. Administrative Appendix 118
18.1. Document History 118
18.2. Acknowledgments 118

© TM Forum 2025. All Rights Reserved.

List of Tables

Table 1: Data Domains 15
Table 2: Summary the AI agent MAS local and global deployment for application domain..... 22
Table 3: Summary of legacy fault management and Intent-Aware Self-Healing ... 24
Table 4: MAS Coordination Type..... 29
Table 5: Sample MAS Coordination Type 29
Table 6: AI Agent MAS Interaction Use Case and Applications Domain..... 29
Table 7: Summary Comparison of Coordination Types 33
Table 8: Comparison of Inference and Simulation 36
Table 9: AI Agent Type and Example Functions 39
Table 10: AI Agent Function uses cases and Application Domains 40
Table 11: Agent Collaboration..... 49
Table 12: MAS Class Hierarchy Turtle File..... 51
Table 13: AI Agent MAS Use Cases and Applications Domain 54
Table 14: AI Agent Types Characteristics Application Domain..... 55
Table 15: AI Agent Triple Store Activity Summary 58
Table 16: TMF633 API Mapping..... 64
Table 17: AI Agents enhancement using TMF633 API 65
Table 18: TMF915 API Common Operations 67
Table 19: TMF915 Use cases and endpoints..... 68
Table 20: TMF915 Common API Operations 69
Table 21: TMF915 API Use Cases Usage 70
Table 22: AD Ontology Representation..... 72
Table 23: Ontology Prefix and Namespace 75
Table 24: Core Class Hierarchy..... 76
Table 25: Key object properties 78
Table 26: Turtle syntax of the complete Ontology 79
Table 27: CFS and RFS Function Transformation 89
Table 28: Diagram Layer Outline Description..... 100
Table 29: AI Agent Module Architecture..... 101
Table 30: Formal Representation Snippet (OWL/Turtle-Aligned) 102
Table 31: Simulation, Emulation and Digital Twin Comparison 105
Table 32: Summary of T-Box and A-Box 106
Table 33: Intent Model 108
Table 34: Intent Field Description..... 109
Table 35: Intent Functional Block..... 109
Table 36: Observability Points..... 110

© TM Forum 2025. All Rights Reserved.

Table 37: Performance Optimization Intent for RAN..... 110
Table 38: TMF Intent Artifact 111

© TM Forum 2025. All Rights Reserved.

List of Figures

Figure 1: Layered Intent AI-Agent Based Service Management Architecture 12
Figure 2: AI Agent MAS Interaction Model across product, service and resource domains 44
Figure 3: Multi-Agent Intent Lifecycle Coordination in Autonomous Network Domains 47
Figure 4: MAS Protege Class Hierarchy (see enlarged Figure 5 in Figure 6)..... 50
Figure 5: Figure 4 Enlarged for Clarity 51
Figure 6: Intent-to-Action Pipeline..... 59
Figure 7: Semantic Execution of Network Intent 60
Figure 8: Protege API's instances in ADs 62
Figure 9: Protégé AD Hierarchy 71
Figure 10: Protégé IMF for AD and AI Agent MAS Ontology..... 75
Figure 11: Three concepts intersect and complement one another in resource-level orchestration. 98
Figure 12: Hybrid Telemetry and Partial Twin Flow 100

© TM Forum 2025. All Rights Reserved.

Executive Summary

The rapid advance of Autonomous Networks (ANs) makes it essential to integrate AI-driven Multi-Agent Systems (MAS) with an intent-based ontology, enabling seamless coordination both within and across Autonomous Domains (ADs). This document presents a structured taxonomy and ontology for AI Agent MAS, aligned with the intent-based management functions (IMF) defined in TMF IG1253 [1], and focuses on the following core capabilities:

- Service intent-based lifecycle management,
- Cross-domain AI agent coordination through intent negotiation,
- Automated policy adaptation and enforcement,
- Fault recovery,
- Service quality assurance,
- Security enforcement,
- Interoperability,
- Intent based network operation optimization,
- Scalability

AI agents operating within a MAS enable an intent-based service management framework by facilitating dynamic adaptability, policy compliance, and operational optimization. When integrated with the IMF, these agents empower autonomous networks with self-orchestration capabilities, reducing operational complexity while improving service reliability. This shift toward intent-aware service management establishes a foundational path toward fully autonomous, AI-driven network operations.

The service management model leverages standardized TMF APIs, [2],[4],[6] and [7] to formalize AI agent interactions, hierarchical intent propagation, and self-optimizing system behavior. Key aspects covered include:

- AI Agent MAS Taxonomy and Ontology - A structured classification and reasoning based on functional roles, semantics model, autonomy, interaction models, and deployment scope [9].
- Intent-Based AI Coordination - Utilizing game-theoretic negotiation strategies and Markov Decision Processes (MDP) [32] for dynamic intent resolution.
- Closed-Loop Self-Optimization - Ensuring adaptive network management across hierarchical intent layers.
- Standardized API Integration - Enabling intent-driven automation in operational systems to enhance scalability and interoperability.

© TM Forum 2025. All Rights Reserved.

By implementing this framework, AN can achieve intelligent, self-governing service management, driving the next phase of AI agent MAS intent-based telecom operations.

© TM Forum 2025. All Rights Reserved.

Introduction

The rapid evolution of Autonomous Networks (ANs) necessitates the integration of AI-driven Multi-Agent Systems (MAS) with an intent-based ontology model to enable seamless coordination within Autonomous Domains (ADs). The AI agent MAS Taxonomy using intent-based ANs, provides a structured classification of AI agents intent-based:

- Resource management provisioning (optimization, fault detection, security enforcement, service assurance)
- Autonomous decision-making (rule-based, goal-driven, learning-based, and hybrid agents)
- Interaction models (autonomous, collaborative, hierarchical)
- Deployment scope (local, global, cross-domain coordination)

The evolution of intent based AN model based on standards TMF1253 [1], TMF1251 [2] and Ericsson Technology Review "Evolving service management toward intent-driven autonomous networks" [3] (Ericsson Review, “Evolving Service Management Toward Intent-Driven Autonomous Networks”, Nov 2024) captures both resource-level and service-level autonomous operations. This document complements the autonomous deployment with AI agents MAS intent-based service management to enable self-adaptive autonomous operations (see Appendix H: Standards-Aligned Intent Execution Template).

At the core of this framework is the Intent Management Function (IMF) and AI agents, which facilitates intent processing, negotiation, and execution within and across ADs. The hierarchical intent structure ensures that business objectives at the product level are translated into service-level and resource-level configurations as shown in Figure 1.

© TM Forum 2025. All Rights Reserved.

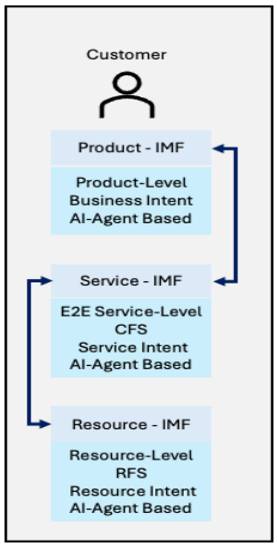

Figure 1: Layered Intent AI-Agent Based Service Management Architecture

Every AD within AN implements IMF and AI agent, which contains an intent manager, which is realized using a closed loop. IMF and AI-agents are the entities that communicate with each other using intent to set requirements. An IMF as well as AI-agent coordinates with other functions within its domain to determine how to achieve the requirements. IMF and AI agent within ADs entities involved in intent-based operation. Every intent has exactly one owner and one handler. IMF plays the role of intent owner if it defines requirements with which another domain needs to comply. In addition, it receives requirements that it must comply with in its own domain, it assumes the role of intent handler. Figure 1 shows IMF and AI-agent in the product, service and resource operation layers. The addition of intent specifications to the service catalog with links to product and service specifications using TMF620 [4] product catalog management and the service and resource catalog management TMF633 [5], TMF921 [8]. This captures intent values during the ordering phase. These intents are product intents when associated with ordering a product and its associated customer-facing services (CFSs). They will be converted into operational intents when ordering the services used to realize products. Intents are communicated within the product order and service order. The end-to-end (E2E) service IMF and AI-agent will utilize the delivery expectation mechanism. A delivery expectation specifies that something needs to be delivered and provides details relating to it. The intent of a product would specify the set of CFSs to be delivered. Further requirements stated in the product-level intent, for example, using property expectations and KPI-based conditions, are distributed into the intents for the respective CFSs. The decomposition of CFSs into applicable resource facing service (RFSs) with associated operational intents is the responsibility of the E2E service IMF located in the service operation layer, which involves functions such as service orchestration.

© TM Forum 2025. All Rights Reserved.

1. Scope and Objectives

This document defines a comprehensive framework for intent-based service management in ANs, leveraging AI-powered MAS operating across ADs. It introduces a structured AI Agent taxonomy and ontology aligned with TM Forum IG1253's [1] IMF and explores how agents interact through standardized TMF APIs [4], [5], [6], and [7].

The scope includes:

- Formalizing **intent** as a semantic expression of goals, constraints, and desired outcomes.
- Modeling AI agents to support the lifecycle of service intent, including discovery, translation, monitoring, and assurance.
- Applying game-theoretic coordination and reinforcement learning for dynamic policy enforcement and cross-domain negotiation.
- Integrating TMF APIs (e.g., TMF620, TMF633, TMF915) for interoperability and automation.

The key capabilities includes:

- Cross-domain AI agent coordination
- Automated policy adaptation and enforcement
- Fault recovery and self-healing
- Service quality assurance
- Security enforcement and zero-touch operations
- Semantic interoperability and scalable orchestration

ADs are presented as self-governing entities responsible for managing services and resources based on real-time environmental feedback and external intent. In this architecture, AI agents use closed-loop automation and knowledge-driven inference to realize intent with minimal human intervention.
By uniting semantic intent, agent reasoning, and telecom standards, this document lays the foundation for intelligent, autonomous, and interoperable service operations.

© TM Forum 2025. All Rights Reserved.

2. AI Agent MAS Taxonomy of AN Service Management

The taxonomy provides a comprehensive framework for understanding the roles, capabilities, and interactions of AI agents in intent-based ANs. These agents work together as part of a MAS to support system optimization, performance management, service quality management, security enforcement, and fault recovery, all driven by the dynamic needs of the system and its users.

The taxonomy of AI agents classifies and structure the agent based on their

- Functional roles
- Decision-making abilities
- Interaction models, and
- Deployment scope.

This follows the principles of taxonomy: categorization, classification, and structuring knowledge in a clear, hierarchical manner.

2.1. AI Agent Functional Role

- Perception & Awareness
    - Role - Gathering data from the environment to inform decision-making.
    - AI agents operate within an adaptive perception loop, where raw telemetry, logs, and signals are collected, processed, and transformed into contextualized information.
    - This transformation is critical: data must be interpreted and structured into information, often via enrichment, correlation, or statistical modeling, before it becomes usable for decision logic
    - Once this context is established, AI agents analyze service performance trends, detect anomalies or threshold violations, and recommend or enact configuration adjustments.
    - While strict real-time responses are constrained by telemetry latency and system architecture, agents operate within adaptive control cycles, responding to changes as observability and infrastructure conditions allow.
    - “Continuously” indicates an ongoing, event-driven process—not instantaneous reaction. “Live insights” better represents the agent’s

© TM Forum 2025. All Rights Reserved.

view of the environment, often driven by streaming telemetry (e.g., gNMI, YANG Push) or periodic updates. “Adaptive cycles” acknowledge the variability in observability precision and enforcement latency across modern, hybrid system domains.

Note: This perception capability is supported by ML models, stream processors, or observability fabrics that turn raw network data into structured, actionable insights. Without this data-to-information transformation, autonomous agents cannot achieve effective, intent-aligned decision-making.

- Functions (AI Agent Perception Layer)
    - Collects rich, multi-layer data from system and service environments, including device and service state, control plane interactions, data plane telemetry (e.g., flow statistics), management plane parameters (e.g., configurations, operational status), and system/application logs. This comprehensive input is necessary to construct the semantic context required for informed decision-making.
    - Executes autonomous actions such as bandwidth allocation, route adjustments, or QoS tuning, based on derived insights, policy constraints, and real-time or near-real-time observability.
    - Interacts with external systems and orchestration layers, triggering workflows or configuration changes via open APIs (e.g., gNMI, RESTCONF, TMF APIs), with closed-loop verification where supported.

Note: Semantic reasoning depends on the ability to synthesize insights across control, data, and management planes, along with time-aligned log data, to infer causality, detect anomalies, or identify trends. AI agents operating with partial or abstracted state alone will lack sufficient context for meaningful autonomous action.

It is important to align the network architecture principle and to distinguish the data domains as shown in the Table 1:

Table 1: Data Domains

Plane	Data Collected
Management Plane	Configuration state, service provisioning data, policy logs
Control Plane	Routing changes, signaling protocols, topology advertisements
Logs & Events	System logs, alarms, user actions, app-level metrics
Data Plane	Traffic volumes, flow latency, drop rates, queue status

© TM Forum 2025. All Rights Reserved.

- Use Cases
    - Intent Management Agent - Interprets customer intent and translates it into technical policies (e.g., "low-latency streaming").
    - Network Optimization Agent - Monitors bandwidth and latency to adjust resources dynamically.
        - in modern intent-based architectures (e.g., TMF IG1253, ETSI ZSM 002), intent is not confined to the business layer. Instead, intent can—and should—exist at multiple levels, including:
            - Business/Product Layer: High-level goals (e.g., improve customer satisfaction)
            - Service Layer: SLA/OLA-aligned objectives (e.g., maintain 20 ms latency)
            - Resource Layer: Operational intent (e.g., maximize link utilization while preserving redundancy)
    - The intent abstraction and refinement occur across all layers — not just at the top. For the Intent across layers
        - Intent Negotiation Agent
          Coordinates multi-layer intent negotiation across business, service, and resource domains. This includes aligning high-level product goals with service-level objectives and resource-level operational intents. Each layer may express its own intent, which is refined or abstracted based on capabilities and context, rather than being reduced to static technical policies.
        - Collaborative Network Management Agents
          Work cooperatively across regions and domains to uphold E2E intent goals, such as maintaining QoS, availability, or latency thresholds. This coordination is guided by layer-specific intents (e.g., resource-level operational intents) rather than direct enforcement of low-level policies, enabling scalable, intent-aware service assurance.
    - Note: Intent is not limited to product or service levels. In modern architectures, each domain may define and negotiate intent based on its scope and abstraction level. For example, a transport network may declare an operational intent to "optimize path diversity under bandwidth saturation" — this is an intent, not just a policy. AI agents facilitate the negotiation and refinement of such intents across

© TM Forum 2025. All Rights Reserved.

domains, maintaining semantic consistency and avoiding manual policy stitching [1],[24], [29].

- Learning & Adaptation
  - Role - Applying ML techniques to improve agent decision-making over time.
- Functions
  - Uses ML models such as RL, neural networks, or adaptive algorithms.
  - Adapts actions based on closed-loop feedback or environmental changes.
- Use Case
  - Learning-based Security Agent: Detects anomalies in network traffic using deep learning and adapts security rules over time.
- Communication & Coordination
  - Role - Enabling agents to collaborate and make coordinated decisions.
- Functions
  - Interacts with other agents within a MAS for collective decision-making.
  - Shares state information and synchronizes operations in a distributed system.
  - Negotiates and shares resources, particularly for cross-domain coordination.
- Use Cases
  - Intent Negotiation Agent - Coordinates intent negotiation between different network domains (product, service, and resource layers).
  - Collaborative Network Management Agents - Work in tandem to maintain QoS standards across different regions.

2.2. AI Agent Decision-Making

- Decision-Making & Reasoning
  - Role - Using logic, policies, or AI-driven models for autonomous decision-making.

© TM Forum 2025. All Rights Reserved.

- Functions
    - Applies reasoning models to determine optimal actions based on the available data.
    - Executes decision-making based on predefined rules, AI models (e.g., RL), or adaptive strategies.
- Use Case
    - Deliberative Agent - A network resource slicing agent that allocates bandwidth to meet SLA requirements.
    - Fault Detection & Recovery Agent - Identifies network failures, uses logic to classify the issue, and determines the corrective action.
- Action Execution & Control
    - Role - Enacting decisions and performing tasks autonomously.
- Functions
    - Executes actions to control the network, such as adjusting configurations, allocating resources, or initiating recovery actions.
    - May trigger workflows or external systems to address performance or security issues.
- Use Cases
    - Network Optimization Agent - Implements network changes to optimize resource allocation (e.g., rerouting traffic based on current conditions).
    - Security Enforcement Agent - Applies security policies to the network, such as activating firewall rules or anomaly detection.
- AI Agents MAS Intent Base Functional Roles
    - Intent Management Agents
    - Intent Owner Agent
        - Assembles and submits intent to an intent handler for processing.
    - Intent Translator Agent
        - Converts high-level business intent into executable network policies.

© TM Forum 2025. All Rights Reserved.

    - Intent Handler Agent
        - Manages the lifecycle of intents, including validation, enforcement, and optimization.
    - Intent Monitoring Agent
        - Observes network/service performance and reports compliance status.

- Network & Resource Management Agents
    - Network Optimization Agent - Dynamically allocates network resources such as bandwidth and routing to improve network performance.
    - Fault Detection & Recovery Agent - Detects network anomalies, triggers corrective actions, and restores service continuity.
    - Security Enforcement Agent - Implements network security policies, such as activating firewalls or detecting network anomalies.
    - Service Quality Agent - Ensures compliance with SLAs and ensures that network services meet QoS parameters.

- AI Agents Autonomy & Decision-Making Capability
    - Reactive Agents (Rule-Based)
    - Operate on predefined conditions and respond accordingly (e.g., firewall agent blocking traffic based on static policies).

- Deliberative (Goal-Driven) Agents
    - Perform goal-based reasoning and action. These agents adapt decisions to internal goals or network conditions (e.g., resource slicing based on SLA constraints).

- Learning Agents
    - Use RL or DL to improve decision-making over time, optimizing network performance based on historical data.

- Hybrid Agents
    - Combine rule-based decision-making with ML models for adaptive behavior (e.g., Self-Optimizing AI Agent in telecom networks operations).

© TM Forum 2025. All Rights Reserved.

2.3. AI Agent Interaction Model

- **Autonomous Agents**
  - Operate independently and autonomously without external intervention. These agents perform tasks like self-healing or proactive fault detection.
- **Collaborative Agents in MAS**
  - Work in coordination to achieve complex goals. These agents cooperate, negotiate, or synchronize operations to manage network resources effectively.
- **Hierarchical Agents**
  - Organized in a layered structure, often used for intent propagation across different layers (Product → Service → Resource levels), with each layer having its specific role in fulfilling business and service-level intents.

2.4. AI Agent Deployment Scope

- Local Agent (Single-Domain see Table 2)
  - Operate within a single AD, typically focused on a specific function such as resource assurance, access optimization, or local service orchestration. While these agents operate locally, they must coordinate with peers within the domain to avoid conflict, contention, or unstable behaviors. Local autonomy is always scoped by shared goals, domain-wide policy awareness, and agent-to-agent coordination protocols.
  - Local AI agents operate within the boundaries of a single AD, often focused on real-time or near-real-time decision-making for edge, resource, or service orchestration. For these agents to function meaningfully, they must be capable of turning raw data (telemetry, logs, metrics) into contextualized information that informs action.
  - Hence, local agents rely on:
    - Domain-bound ontologies (e.g., YANG models, TMF SID extensions) that define the structure and meaning of local data.
    - Pre-trained or reinforcement-learned models tailored to the local network behavior or service model.
    - Semantic enrichment mechanisms to interpret state in a machine-actionable way.

© TM Forum 2025. All Rights Reserved.

- Edge AI Agents - Function within edge environments to optimize local resource utilization (e.g., compute, routing, load balancing). However, they cannot operate in isolation—edge agents must coordinate with other local agents to ensure consistency, prevent action loops, and respect shared resource constraints. Uncoordinated decisions may result in systemic instability (e.g., service thrashing or network congestion). Therefore, intra-domain coordination mechanisms—such as local agent consensus, shared state repositories, or intent propagation—are essential to maintain predictable, stable behavior.

Note: Edge agents must balance fast, localized decision-making with collaborative domain behavior. They may leverage domain-level coordinators or shared knowledge graphs to align on global objectives (e.g., avoiding overload or resource starvation). This mirrors practices in distributed systems like SDN controllers, where consistency and coordination models are critical for stability 24, 30.

- Global Agents (Cross-Domain see Table 2)
  - Global agents operate across multiple ADs—e.g., access, core, and cloud—and are responsible for cross-domain intent negotiation, policy harmonization, and resource balancing. These agents must reason over diverse, heterogeneous knowledge base
  - Coordinate operations across multiple network domains, such as across access, transport, core, and cloud layers.
  - Global AI Agents - Facilitate cross-domain intent negotiation, balancing resources across various domains (e.g., access, transport, core, cloud) to ensure global optimization and SLA adherence.
- This is typically achieved through:
  - Ontology alignment or federation, where each domain's ontology is semantically linked to others through mappings or transformation rules.
  - Use of SKOS (Simple Knowledge Organization System) or upper ontologies to define shared vocabularies (e.g., service categories, resource capabilities, QoS classes).
  - Shared conceptual graphs or knowledge hubs that serve as a mediation layer across AI agents operating with different internal models.
- Thus, global agents require (see Table 2):
  - Access to ontology abstraction layers for semantic interoperability.

© TM Forum 2025. All Rights Reserved.

- Mechanisms for semantic translation or reconciliation, such as concept mapping engines or linked data models.

Table 2: Summary the AI agent MAS local and global deployment for application domain

Scope	Characteristics	Use Case	Applicable Domain	Knowledge Foundation
Local Agents (Single-Domain)	Operate within a single AD; responsible for localized decision-making and actuation.	Edge AI Agent optimizes local compute and resource utilization.	**Resource-Level AD**	**Requires domain-specific ontologies or custom ML models to turn data into actionable information.**
Global Agents (Cross-Domain)	Coordinate across multiple ADs to reconcile service-level goals and policies.	Inter-Domain Intent Negotiation Agent balances resources across access, core, and cloud.	**Product, Service & Resource-Level AD**	**Requires shared conceptual models built on ontology abstraction (e.g., SKOS), enabling cross-domain semantic alignment.**

The taxonomy provides a comprehensive framework for understanding the roles, capabilities, and interactions of AI agents in intent-based autonomous networks. These agents work together as part of a MAS to support network optimization, performance management, service quality management, security enforcement, and fault recovery, all driven by the dynamic needs of the network and its users.

© TM Forum 2025. All Rights Reserved.

3. AI Agent MAS Intent-Based Interaction Model

AI agents MAS coordination refers to the mechanisms to enable multiple AI agents to work together to achieve shared or individual goals in a decentralized system. In the context of intent-based ANs, different coordination types ensure effective resource allocation, conflict resolution, and service optimization.

Selecting the appropriate MAS intent-base coordination depends on scalability, optimization objectives, fault tolerance, and decision speed.

A hybrid approach combining hierarchical, distributed, and contract-based coordination is often necessary for robust, real-time AN service management. The selected appropriate coordination would ensure optimal service performance and seamless cross-domain interactions. Below are the primary types of multi-agent coordination.

3.1. Autonomous AI Agents

Autonomous AI Agents operate independently within a defined policy and intent framework, making decisions and executing actions without external intervention. Their intelligence stems not just from detecting failures, but from recognizing non-conformance to desired outcomes — even when systems appear “healthy” from a fault-management perspective.

- Use case, self-healing - AI agent detecting and recovering from failures.
    - A common use case is proactive service assurance in an SD-WAN environment. For example:
        - The network path is operational and fault-free.
        - However, jitter exceeds the customer’s threshold, or latency increases beyond acceptable application tolerance.
        - The AI agent autonomously switches traffic to a lower-cost or higher-performance path that still meets the SLA/OLA — before the user perceives an issue or complains.
        - In this model, nothing is broken, but the system is not aligned with operational or business intent. This represents a shift from failure-driven remediation to intent-driven optimization.
    - Autonomous agents therefore:
        - Continuously monitor for deviations from policy, SLA/OLA, or real-time service metrics.
        - Use predictive analytics or streaming telemetry to detect degradations before they cause failures.

© TM Forum 2025. All Rights Reserved.

- Act in proactive cycles, triggering corrective workflows (e.g., policy reapplication, path rerouting, workload reallocation) without waiting for fault alarms or service tickets.

Key distinctions are summarized in the Table 3.

Table 3: Summary of legacy fault management and Intent-Aware Self-Healing

Traditional Fault Management	Intent-Aware Self-Healing
Triggered by system failure or alarms	Triggered by deviations from expected behavior
Reactive: “Something is broken”	Proactive: “Something may fall short of customer expectations”
Often results in user impact before action	Acts before user complaints occur
Limited to equipment/service faults	Includes performance drift, resource contention, or SLA violations

3.2. Collaborative AI Agents

Collaborative AI Agents work together to solve complex tasks through coordination and negotiation.

- Use case, service-level coordination – AI agents negotiating between product, service, and resource layers.

3.3. Hierarchical AI Agents

Hierarchical AI Agents organized in a layered structure for intent propagation.

- Product-Level Agent
  - Defines customer-facing intent.
- Service-Level Agent
  - Converts into service-level requirements.
- Resource-Level Agent (within hierarchical agent)
  - Maps service requirements to network configurations.
  - Resource-level AI agents operate at the infrastructure layer of the hierarchy and are responsible for bridging service-level intent with the real-time, fine-grained operational view of the network. Their role extends far beyond issuing configuration commands — they are intelligent observers, interpreters, and synthesizers of the entire operational state.

    Key Functions:
    - Ingest and interpret operational state data across the control, data, and management planes (e.g., telemetry, queues, link status, CPU/memory utilization, interface KPIs).

© TM Forum 2025. All Rights Reserved.

- Map service requirements to resource capabilities, not just through static configuration, but through real-time awareness of load, fault domains, topological availability, and failure zones.
- Align state data with the underlying schema (e.g., YANG models or TMF resource models) to transform raw telemetry into structured, meaningful information.
- Enrich and correlate localized data with inputs from other agents or domains to form a network-wide semantic view—essential for accurate intent fulfillment, assurance, and self-optimization.
- While configuration change may still be executed these agents primarily operate in a sense-understand-act loop, where the "understand" phase is anchored in data-model-driven reasoning rather than template-driven automation. In modern Tier 1/2 networks, configuration is already handled by policy engines; the agent's value lies in augmenting policy decisions with continuous situational awareness.

3.4. Centralized Coordination Agents

A single coordinating entity (central agent or controller) manages the interactions between AI agents, ensuring optimized decision-making.

- Use cases in Intent-based networks operation:
  - A central intent coordinator agent collects intent requests from multiple ADs and distributes policies to domain-specific agents.
  - Used in hierarchical intent propagation, where higher-level intent managers direct lower-level agents in service and resource orchestration.

- Advantages:
  - Ensures global optimization with full system visibility.
  - Simplifies conflict resolution between agents.

- Disadvantages:
  - Single point of failure.
  - Limits scalability in highly dynamic, large-scale networks.

© TM Forum 2025. All Rights Reserved.

3.5. Distributed Coordination Agents

Agents coordinate without a central authority, relying on peer-to-peer communication to share information and make decisions.

- Use cases in intent-based networks operations
    - AI Agents in a multi-domain environment negotiate resource allocation through decentralized intent-based negotiations.
    - Service-Level Agents adapt dynamically to changes in demand without waiting for higher-level approvals.
- Advantages:
    - Improves scalability and fault tolerance.
    - Allows local optimization and faster decision-making.
- Disadvantages:
    - Requires efficient communication protocols to avoid inconsistencies.
    - Can lead to suboptimal decisions due to limited visibility.

VI. Hierarchical Coordination Agents

A multi-layered coordination approach where AI agents are organized into hierarchical levels, with higher-level agents setting policies for lower-level agents.

Example in Intent-Based Networks:

- Product-Level AI Agents define high-level business intent.
- Service-Level AI Agents translate intent into service configurations.
- Resource-Level AI Agents render network changes via vendor-specific YANG or CLI template/ policy engines. End-to-end declarative intent at this layer is still limited by heterogeneous device models (ongoing IETF NMOP & OpenConfig efforts aim to close the gap)

- Advantages:
    - Ensures structured decision-making across intent layers.
    - Supports intent propagation from high-level business goals to low-level network actions.
- Disadvantages:
    - Increased latency due to multi-layered processing.
    - Requires efficient synchronization between layers.

© TM Forum 2025. All Rights Reserved.

3.6. Market-Based Coordination AI Agents

Agents act as buyers and sellers in a virtual marketplace, negotiating for resources based on supply and demand.

- Use case in intent-based networks operations:
    - Bandwidth allocation agents bid for network capacity based on service demand.
    - AI-driven intent negotiation agents dynamically price network services based on congestion levels.
- Advantages:
    - Provides economic efficiency by aligning supply and demand.
    - Enables dynamic resource allocation in large-scale networks.
- Disadvantages:
    - Requires well-defined pricing and auction mechanisms.
    - Can be computationally intensive.

3.7. Contract-Based Coordination AI Agents

Agents form agreements or smart contracts to ensure compliance with predefined rules and service-level guarantees.

- Use case in intent-based network operations:
    - AI Agents negotiating SLAs in cross-domain services using blockchain-based smart contracts.
    - Intent handler agents ensuring policy adherence based on formalized agreements.
- Advantages:
    - Provides transparency and verification.
    - Reduces disputes by formalizing agent commitments.
- Disadvantages:
    - Can introduce complexity in contract enforcement.
    - Requires trust in contract execution mechanisms.

© TM Forum 2025. All Rights Reserved.

3.8. Coalition-Based Coordination AI Agents

Groups of agents form temporary or long-term coalitions to achieve collective objectives.

- Use cases in intent-based networks operations:
    - Service federation agents coordinate across different telecom operators to ensure seamless roaming and interconnectivity.
    - Security enforcement agents collaborate across multiple domains to detect and mitigate threats.
- Advantages
    - Enhances efficiency in complex multi-agent environments.
    - Enables cooperative resource sharing and policy alignment.
- Disadvantages
    - Requires mechanisms for coalition formation and dissolution.
    - Can lead to conflicts if coalition goals diverge.

Game-Theoretic Coordination Agents

Agents make strategic decisions based on mathematical game theory models to optimize their actions in competitive or cooperative settings.

- Use case in intent-based networks operations
    - AI agents optimizing network slicing through Nash equilibrium-based strategies.
    - Intent negotiation agents balancing service quality vs. resource costs using multi-agent reinforcement learning.
- Advantages
    - Enables optimal decision-making in competitive environments.
    - Supports both cooperative and adversarial agent interactions.
- Disadvantages
    - Computational complexity increases with agent interactions.
    - Requires accurate modeling of agent preferences and payoffs.

The multi-agent coordination types are shown in Table 4, function coordination type is summarized in Table 5, and Table 6 shows a summary of interaction AI Agent MAS use cases for a given application domain.

© TM Forum 2025. All Rights Reserved.

Table 4: MAS Coordination Type

Coordination Type	Centralized	Distributed	Hierarchical	Market -Based	Contract -Based	Coalition -Based	Game-Theoretic
Scalability	Low	High	Medium	High	Medium	High	Medium
Fault Tolerance	Low	High	Medium	Medium	High	High	Medium
Optimization Focus	Global	Local	Layered	Economic	Compliance	Collaborative	Strategic
Decision Speed	Moderate	High	Medium	High	Medium	Medium	Medium
Implementation Complexity	Medium	High	High	High	Medium	High	High

Table 5: Sample MAS Coordination Type

Category	Function	Coordination Type
Intent Agents	Handle user intent translation & validation	**Negotiation-based**
Optimization Agents	Optimize network, compute, and service resources	**Game-theoretic**
Monitoring Agents	Observe system states & report anomalies	**Distributed RL**
Fault Recovery Agents	Detect and resolve faults autonomously	**Multi-agent RL**
Security Agents	Detect intrusions & enforce security policies	**Multi-agent coordination**

Table 6: AI Agent MAS Interaction Use Case and Applications Domain

Type of Interaction	Characteristics	Example Use Case	Applicable Domain
Collaborative (MAS)	Work together to solve complex tasks via coordination and negotiation.	Service Coordination Agents negotiate between product, service, and resource layers.	**All ADs**
Hierarchical Agents	Organized in a structured intent propagation model.	Product-Level Agent defines customer intent → Service-Level Agent converts intent to technical specs → Resource-Level Agent implements network configurations.	**All ADs**
Autonomous Agents	Operate independently, making decisions without external intervention.	Self-Healing Network Agent detects and recovers from failures.	**Service & Resource-Level AD**

© TM Forum 2025. All Rights Reserved.

3.9. AI Agent Simulation & Inference as a Coordination Mechanism

In an intent-based AN, AI agents may coordinate through simulation and inference, where agents anticipate the state of the network and the likely actions of their peers—not through explicit communication, but through predictive modeling and contextual awareness.

However, the practical realization of this coordination model is currently limited by the way network data is collected and filtered. Today's operational environments often drop or aggregate events at collection points to make the data manageable for human operators. While effective historically, this introduces significant semantic loss—removing the granularity and context required for accurate prediction or inferencing.

For example, important network anomalies may never be recorded in a usable form due to log suppression, low polling frequency, or event deduplication. This directly limits the agent's ability to reason about what has occurred—or to simulate what may happen.

AI agents in an intent-base AN simulate possible network states and infer optimal decisions based on predictive models. This allows agents to coordinate indirectly by anticipating the actions of other agents and making proactive adjustments.

AI agent simulation & inference is an implicit and predictive form of coordination where AI agents anticipate, simulate, and infer actions rather than explicitly communicating with one another. This type of coordination is highly effective in dynamic, complex, and large-scale networks operations where real-time learning and adaptability are critical. It can be used in combination with other coordination mechanisms (e.g., game-theoretic and distributed coordination) to enhance autonomous decision-making, proactive optimization, and self-healing capabilities in intent-based networks operations.

To truly enable simulation and inference:

- Networks must adopt machine-oriented observability architectures, where telemetry, logs, and events are semantically enriched, time-aligned, and retained in full fidelity—even if not all data is immediately acted upon.
- This requires architectural shifts such as:
    - Stream-based telemetry collection (e.g., gNMI, YANG Push)
    - Event correlation layers with semantic tagging
    - Data mesh or fabric models that decouple collection from human interpretability
- Simulation and inference remain a powerful future-state capability for multi-agent coordination—particularly in large-scale, dynamic

© TM Forum 2025. All Rights Reserved.

environments where policy-based coordination is too slow or rigid. However, to be effective, it must be paired with:

  - High-fidelity observability infrastructure
  - Semantic enrichment pipelines
  - Longitudinal data retention for training predictive models
- Without these foundations, simulation may devolve into approximation, and inference into speculation.
- This coordination mode is best seen as complementary to others (e.g., game-theoretic, hierarchical, distributed), and its maturity is tied closely to how well networks evolve to support machine-driven insight rather than human-centric event reduction.

Key Characteristics of Simulation & Inference Coordination

- Predictive Decision-Making
  - AI agents use simulation models (e.g. digital twins, however, a true digital twin of the network layer, with real-time bi-directional fidelity, does not yet exist in current production telecom environments. Monte Carlo simulations) to evaluate potential network scenarios before making real-world decisions. While the term “digital twin” is often used aspirational, in practice, what exists today are model-driven abstractions, partial network simulators, or domain-specific representations (e.g., traffic maps, intent overlays, topology graphs) that support decision-making in constrained scopes.
  - Predictive Decision-Making
    - AI agents leverage simulation-assisted inference techniques, including topology models, historical pattern matching, and constrained simulations (e.g., Monte Carlo methods), to evaluate possible outcomes before executing real-world decisions. While full digital twins of the network layer are not yet realizable, agents can operate with partial models or abstracted environment representations to simulate routing, load balancing, or failover scenarios.
    - Use Case: A network optimization agent evaluates alternative routing paths across available infrastructure using policy, telemetry, and topological constraints to infer the most optimal configuration, minimizing service impact without requiring full system reconfiguration.
    - This coordination mechanism is goal-oriented: rather than relying on complete real-time replicas of the network (digital twins), agents work with bounded knowledge graphs, partial topology models, and performance envelopes to reason over likely outcomes and adjust accordingly. Over time, as observability and semantic modeling improve, the fidelity of

© TM Forum 2025. All Rights Reserved.

such simulation will increase — but current architectures still require abstraction and inference rather than twin-like precision.

- Probabilistic Coordination
    - Unlike explicit communication-based coordination (e.g., contract-based or game-theoretic models), simulation & inference rely on probabilistic models to anticipate the behavior of other agents.
    - Use case, traffic prediction agent infers congestion patterns based on historical data and informs load balancing agents to adjust resource allocation.
- Implicit Coordination via Shared Knowledge
    - Instead of directly negotiating, agents infer their actions by referencing a shared knowledge base or intent-driven policies.
    - Use case, intent negotiation agents infer the most likely service demand fluctuations and autonomously adjust their negotiation strategies.
- Cognitive & Learning-Based Coordination
    - AI agents continuously learn from historical events and adapt their decision-making to optimize future coordination.
    - Use case, fault recovery agent simulates different failure scenarios and proactively deploys mitigation strategies before actual incidents occur.

Table 7 shows a Comparison of Simulation & Inference with Other Coordination Types

© TM Forum 2025. All Rights Reserved.

Table 7: Summary Comparison of Coordination Types

Coordination Type	Explicit Communication?	Predictive?	Adaptive Over Time?	Scalability	Example Use Case	Interaction Type	Interaction Pattern	API Consideration
Centralized	Yes	No	Low	Medium	Top-down network control	Command Response	One-to-many	Requires a centralized orchestration API (e.g., TMF641 Service Order, TMF640 Service Activation); suitable for top-down workflows and intent translation.
Distributed	Yes	No	High	High	Peer-to-peer agent negotiation	Peer negotiation	Many-to-many	Requires peer-discovery and high-fidelity, event-stream APIs (e.g., gNMI/YANG Push, gRPC-based pub-sub). Legacy alarm chains (TMF701) are acceptable only for backward compatibility—semantic context is preserved by avoiding NOC filtering and keeping raw events in the data fabric for AI reasoning. State-synchronization and schema alignment remain essential.
hierarchical	Yes	No	Medium	Medium	Intent propagation	Cascading delegation	Parent-child	API support for resource advertisement, auction protocols, and bid

©TM Forum 2025. All Rights Reserved.

Coordination Type	Explicit Communication?	Predictive?	Adaptive Over Time?	Scalability	Example Use Case	Interaction Type	Interaction Pattern	API Consideration
					across domains			evaluation (e.g., GraphQL or TMF620 Product Catalog extensions).
Market-Based	Yes	No	High	High	Bandwidth bidding for 5G slices	Bidding & auction	Competitive-many-to-one	API support for resource advertisement, auction protocols, and bid evaluation (e.g., GraphQL or TMF620 Product Catalog extensions).
Contract-Based	Yes	No	Medium	Medium	SLA-based AI agent coordination	SLA negotiation	Offer-accept-sign	Requires policy negotiation APIs (e.g., TMF652 Agreement, TMF645 Service Qualification); agents negotiate SLAs based on service specs.
Coalition-Based	Yes	No	Medium	High	AI agents forming alliances for resource sharing	Joint agreement	Dynamic group formation	APIs for group membership, resource pooling, and shared SLA enforcement; may involve trust/auth delegation.
Game-Theoretic	Yes	Yes	Medium	Medium	Nash equilibrium-based intent negotiation	Strategy-based negotiation	Iterative, bounded	Requires state-sharing and payoff estimation APIs; used for conflict resolution with

© TM Forum 2025. All Rights Reserved.

Coordination Type	Explicit Communication?	Predictive?	Adaptive Over Time?	Scalability	Example Use Case	Interaction Type	Interaction Pattern	API Consideration
								optional predictive modeling integration.
Simulation & Inference	No	Yes	High	High	AI agents preemptively simulating network behavior	IImplicit, non-communicative	AI agents' reason over anticipated state or policy outcomes rather than direct messaging.	Requires **semantically enriched information** derived from topology, policy, and telemetry models. **Behavioral simulation** (e.g., traffic convergence) is limited due to lack of vendor-specific modeling. **Policy/intent simulation**, however, is practically useful in bounded contexts—such as validating path diversity, testing "what-if" scenarios, or checking preconditions for service deployment—using topological overlays (e.g., via TMF686). Simulation may enhance inference but is not required. Inference operates on structured knowledge representations built from observed or declared states.

© TM Forum 2025. All Rights Reserved.

Simulation & Inference is a non-explicit coordination method where agents act based on internal reasoning or model-based expectation of other agents' behavior. However, inference does not require simulation. It requires interpretable, context-aware information derived from semantically modeled data. Simulation adds value by projecting potential future states, but meaningful inference can occur purely from enriched operational context (e.g., network logs, policy states, KPI trends).

To enable robust inference:

- AI agents must operate on semantically structured data (ontologies, telemetry mapped to models, normalized KPIs).
- Contextual awareness is essential — data must be turned into information, aligned with known schemas, and positioned within the agent's intent or policy domain.

The **AI reasoning pipeline** (Data → Information → Knowledge → Decision). One cannot infer from raw data without semantic grounding. A comparison of Simulation is inference is outlined in Table 8.

Table 8: Comparison of Inference and Simulation

Aspect	Inference	Simulation
Relation	Can occur without simulation	Can enhance inference if model is sound
Purpose	Derive conclusions from known or observed information	Explore outcomes under hypothetical scenarios
Requirement	Semantic understanding of data (information)	Model of system behavior or dynamics

3.10. Self-Healing in Intent-Based AI Agent MAS Service Management

Self-healing is a foundational capability in ANs that enables systems to detect, diagnose, and correct faults without human intervention. In the context of AI Agent MAS operating within an intent-based service management framework, self-healing extends beyond traditional fault remediation to include predictive detection, automated correction, and resilient recovery across the service lifecycle—from order capture to service fulfillment and assurance.

Within the order-to-service provisioning and management platform, self-healing agents continuously monitor workflows, service orchestration chains, and underlying infrastructure states. When anomalies or failures are detected—such as stalled orders, policy violations, or resource allocation mismatches—these agents autonomously trigger predefined or learned corrective actions. These actions may include retrying failed operations, reallocating resources, modifying

©TM Forum 2025. All Rights Reserved.

orchestration paths, or escalating intent modifications to upstream controllers. This proactive remediation reduces the Mean Time to Repair (MTTR), increases service availability, and minimizes manual interventions.

AI agents leverage ML models and historical telemetry to identify early warning indicators of service degradation or workflow disruption. This predictive capability is tightly coupled with intent resolution logic, allowing the network to not only react to faults but to anticipate them and act preemptively. For example, if a service intent specifies low-latency delivery for a critical application, and the system detects increasing latency trends along a path, the AI agent can automatically reconfigure the route or reassign workloads to alternative slices or edge nodes.

Furthermore, self-healing is implemented in closed-loop control cycles supported by TMF-compliant APIs, ensuring policy-aligned enforcement and end-to-end visibility. These control loops monitor KPIs and service health indicators in real time, feed insights back into decision policies, and orchestrate corrective actions at both local (intra-domain) and global (cross-domain) levels.

By embedding self-healing into the AI Agent MAS architecture, service providers can achieve true zero-touch operations, reduce OPEX, and improve customer experience. As networks scale and diversify with emerging use cases like IoT, URLLC, and private 5G, self-healing ensures operational continuity and intent fulfillment across dynamic and distributed service environments.

© TM Forum 2025. All Rights Reserved.

4. AI Agent MAS Intent-Based Service Management

The ADs are distributed across different layers of the operation stack, including product, service, and resource layers. Within these layers, multiple ADs operate either in parallel or in subordinate hierarchies, each responsible for a specific operational scope within the AN. AI agent MAS Intent based management within the ADs facilitate dynamic coordination and decision-making. Each AD utilizes AI Agents MAS to process, enforce, and adapt intent dynamically. AI-based IMF oversee domain-specific responsibilities, ensuring compliance with intent requirements and optimizing performance through a continuous feedback loop. AI agent MAS enable cross-domain collaboration, ensuring effective coordination between intent owners, intent translators, and intent handlers to maintain an optimized network state.

4.1. AI Agent MAS Intent-Based Functional Entities

Intent-based service management leverages AI agents operating within a MAS architecture to dynamically adjust service and resource requirements across domains. These agents function within the IMF, enabling autonomous coordination aligned with business, service, and operational objectives. AI agent functional entities include:

- **Intent Owner Agent**
  - Defines product or operational goals at a high level, including performance, compliance, and assurance expectations.
- **Intent Translator Agent**
  - Converts high-level, declarative business goals—such as those defined by NorthStar intent models in AN—into context-aware, machine-readable rules or policies, where applicable.
  - The Intent Translator Agent acts as a semantic and operational bridge between high-level, user-declared business/product intent and actionable service or network policies. Its function includes:
    - Parsing high-level declarative goals (e.g., “Ensure service uptime > 99.99%”)
    - Mapping intent attributes and constraints into semantically grounded ontology concepts
    - Generating targeted policies, configurations, or triggers for downstream AI agents (e.g., Handler, Monitoring) Intent Handler Agent
    - Enforces the real-time execution of operational intent, ensuring compliance with declared goals. This agent can work with or without explicit policy-based control, depending

© TM Forum 2025. All Rights Reserved.

on whether the environment supports imperative or declarative execution.

- **Intent Monitoring Agent**
  - Observes service behavior and environmental conditions. Crucially, this agent includes embedded analytics capabilities—not just passive monitoring—to perform localized reasoning, detect deviations, correlate data, and generate adaptive recommendations. It can also ingest insights from external data mesh/fabric components (e.g., NWDAF or other analytic services) to enhance situational awareness.
- **Intent Negotiation Agent**
  - Facilitates cross-domain alignment by harmonizing possibly conflicting intents across business, service, and resource layers. It supports coordination among autonomous domains through negotiation, trade-off evaluation, and context-aware adjustment.

These AI agents collaborate within a MAS framework, enabling autonomous coordination across business, service, and resource layers. Each AI agent operates based on real-time data, continuously refining and updating intent to align with evolving network conditions. For example, Table 9 depicts AI agent type and its function, and Table 10 shows AI agent function use cases within the applicable domain.

These agents collaborate through declarative coordination, consistent with the NorthStar intent paradigm in AN framework, where outcomes are specified without prescribing the implementation path. The MAS structure supports both service intent (as defined in TMF IG1358 and RFC 9315, typically tied to service fulfillment workflows) and operational intent, which focuses on runtime assurance, performance, and resiliency objectives. For clarity, service intent governs what a service must achieve (e.g., "provision this slice with 20 ms latency"), whereas operational intent governs how it should behave and adapt over time (e.g., "maintain latency below threshold and recover if it breaches"). For example, Table 9 outlines the roles of each AI agent, while Table 10 illustrates typical use cases across fulfillment and assurance scenarios, reflecting both service and operational perspectives.

Table 9: AI Agent Type and Example Functions

Agent Type	Function	Example
Security Agent	Monitors threats and enforces security policies	AI-powered anomaly detection
QoS Agent	Ensures SLA and QoS compliance	5G network slicing coordinator
Network Optimization Agent	Adjusts routing and bandwidth dynamically and	SDN-based traffic engineering that executes service or operational intent; dynamically adjusts configurations such as routing, QoS, bandwidth, or performs

© TM Forum 2025. All Rights Reserved.

Agent Type	Function	Example
		technology transitions (e.g., 4G to 5G, IPv4 to IPv6).
Multi-Agent Coordinator	– Coordinates cross-domain intent – Negotiates between agents for resource allocation	Negotiates path characteristics between access and core domains to meet cross-slice latency target.
Intent Translator	Converts business intent into technical policies	AI-based policy recommendation and defines high-level business, service, or operational goals (e.g., assurance levels, compliance expectations).
Intent Handler	Manages intent lifecycle	TMF921 Intent API manager that translates declarative intent into actionable rules or configurations; aligns intents to domain-specific semantics.
Fault Recovery Agent	– Detects and mitigates failures, – Self-Healing Agent *(sub-role or composite)*	– Self-healing network agent. – Detects and remediates service or application degradations, not just hardware faults. Can trigger adaptive workflows (e.g., rerouting, autoscaling, service restarts) based on observed KPIs, user experience metrics, or telemetry trends—even in the absence of a declared fault. Integrates data from observability pipelines and analytic models to anticipate service-level risks. – Note: Self-healing operates across layers—reacting to faults, but also proactively responding to degradation, anomaly patterns, or threshold drift, especially where **no** discrete equipment failure is reported.

Table 10: AI Agent Function uses cases and Application Domains

Agent Name	Function	Use Case	Applicable Domain
Intent Owner Agent	Gathers and composes customer-driven or system-generated intents.	A customer requests low-latency service, triggering an intent assembly.	Product-Level AD
Intent Translator Agent	Converts high-level business intents into executable policies and translates high-level business objectives into technical policies	Translates a 5G slicing request into QoS parameters. e.g., convert "maximize user video quality under 100ms latency" into SLA targets or OLA thresholds.	Product to Service-Level AD

© TM Forum 2025. All Rights Reserved.

Agent Name	Function	Use Case	Applicable Domain
Intent Handler Agent	Manages lifecycle: validation, enforcement, optimization. Executes operational intent autonomously	Ensures an intent remains compliant with OLA and SLA. Adjusts bandwidth, reroutes traffic, or reallocates compute during congestion or maintenance.	Service & Resource-Level AD
Intent Monitoring Agent	– Monitors service and network performance, e.g., latency, jitter, response time, app availability. – Ensures compliance with intent expectations. – Analyzes and validates intent fulfillment. – Triggers remediation actions – Triggers self-healing actions based on operational degradation – Self-healing is **not solely fault driven**. It operates based on **deviation from intent**, loss of service quality, or even user experience signals—thus requiring integration with telemetry, analytics, and service-level observability.	– Detects QoS degradation and adjusts bandwidth allocation and monitors both service and network performance, collects KPIs, – Evaluates intent compliance and detects drift or anomalies from expectations. – Checks whether performance metrics align with declared goals (OLAs/SLOs). – Based on degradation of application health (not just equipment faults), triggers retry or scaling. – Detects performance anomalies in services (e.g., increased latency, user complaints, or app errors) and launches corrective actions (e.g., restarting services, triggering failover, reallocating resources)—even when no fault alarms are raised by Fault Management.	– Service & Resource-Level – Service & Application Layer – Operations /Observability

Note:

- SLA vs. OLA: SLAs are external, business-facing contracts; operational execution aligns more closely with OLAs (Operational Level Agreements) or SLOs (Service Level Objectives).
- Performance Monitoring: Includes both network-level metrics (e.g., bandwidth, packet loss) and service-level indicators (e.g., application latency, user experience metrics).
- QoS Implications: QoS changes (e.g., DSCP classification) often require network-wide coordination, since hop-by-hop enforcement affects end-to-end outcomes.

© TM Forum 2025. All Rights Reserved.

4.2. AI Agent MAS Intent-Based Service Management AD

AI agent MAS intent based for Service management follows a structured eTOM TMF GB921E approach:

- Product-Level AI Agents
  - Ensure business intent aligns with customer expectations.
- Service-Level AI Agents
  - Convert business objectives into service-oriented policies.
- Resource-Level AI Agents
  - Manage network and infrastructure configurations.
  - Coordinate and apply configuration changes to network and infrastructure elements. While current implementations rely heavily on intent-to-policy mappings and static templates, especially in heterogeneous or fixed-function domains, emerging models such as model-driven orchestration, semantic normalization, and digital twins aim to enable deeper automation and autonomous adaptation in resource-layer control [24],[25] and [26].
- Real-time Cross-Domain Coordination
  - AI Agents coordinate intent across business, service, and resource layers. While "real-time" coordination is a long-term goal, current deployments face practical challenges in achieving low-latency, end-to-end observability. Legacy telemetry protocols (e.g., SNMP, CLI polling) lack timestamping and impose delayed, asynchronous visibility, making cross-device and cross-domain correlation difficult. For example, data polled every 15 minutes can introduce blind spots when validating time-sensitive outcomes.

  - Emerging telemetry protocols such as gNMI, YANG Push, and gRPC streaming are designed to address these limitations by supporting event-driven, time-stamped telemetry. However, the co-existence of legacy and modern infrastructure—which is expected to persist for the next decade—means that AI agent coordination must incorporate data freshness tracking, confidence scoring, and temporal reconciliation strategies to function effectively in hybrid environments [27], [28].
  - Bridge the intent-analysis gap, even in partially observable conditions:
    - Use of probabilistic reasoning or confidence thresholds for making decisions under uncertainty.
    - Incorporation of temporal models (e.g., sliding windows, Bayesian time-series) to model lagging data.
    - Coordination at different temporal layers (e.g., batch-policy updates for legacy; event-driven for programmable domains).
  - AI Agents communicate intent across business, service, and resource layers.

© TM Forum 2025. All Rights Reserved.

- Autonomous Policy Enforcement
    - AI agents enforce operational and service-level goals by translating declarative intents into executable configurations and capability-aligned policies. While current enforcement is often constrained by static product and resource models, AI mediation agents can support intent negotiation, propose service variants, or raise early conflict signals. This minimizes reactive negotiation and helps evolve toward intent-aware, adaptive service orchestration that blends declarative outcomes with existing product definitions.
    - intent assurance agents can continuously validate whether customer goals are met, and when not, recommend policy adaptations or propose enriched offers via dynamic product exposure APIs (e.g., TMF620 + TMF921).
    - Future models may incorporate constraint solvers, utility optimization functions, or graph-based policy overlays to reduce the policy-intent mismatch space.

- Proactive Optimization & Adaptation
    - AI agents continuously refine network configurations based on real-time data.

    - Adaptive Optimization & Policy-Driven Adaptation
        - AI agents refine network configurations based on streaming insights, policy thresholds, and observed trends, enabling proactive adjustment of service parameters and resource allocations.
        - While strict real-time operation is rarely feasible, agents operate within responsive cycles aligned with system telemetry and control capabilities.

4.3. Deployment Scenario

This section covers AI Agent MAS Interaction Model Across Domains as shown in Figure 2.

- We assume a “horizontal cross-layer structure” that watches over the vertical flow.
- It interfaces with:
    - Intent Owner to capture business/product goals
    - Intent Handlers to trigger configurations or decisions
    - Intent Monitoring Agents to detect issues
    - Fault Recovery Agent to respond to disruptions
- It may be implemented as a distributed coordination layer or central MAS structure.

© TM Forum 2025. All Rights Reserved.

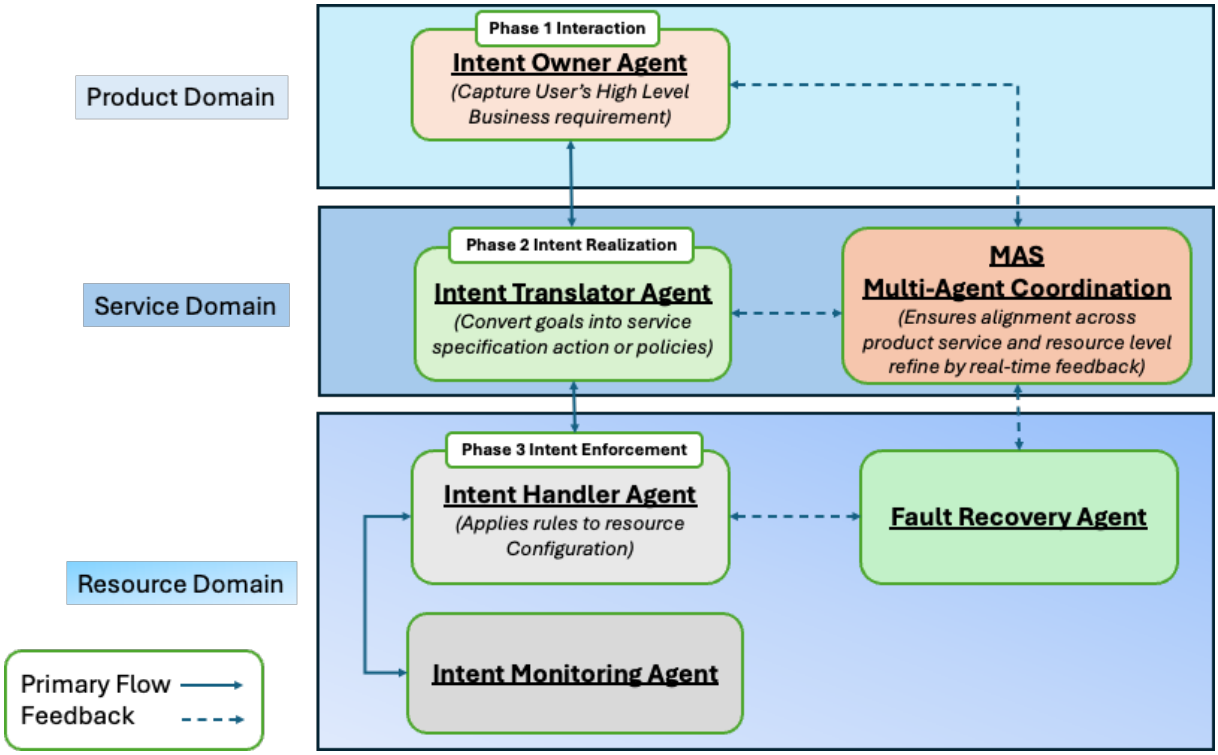

Figure 2: AI Agent MAS Interaction Model across product, service and resource domains

- Agents are mapped clearly to Product, Service, and Resource layers
- Figure 2 displays vertical flow: from Intent Expression (Product layer) → Service decomposition → Execution (Resource layer)
- Multi-Agent Coordinator shown bridging across domains
- Clearer placement of Intent Monitoring Agent and Fault Recovery Agent

4.3.1. Multi-Agent System (MAS) Coordination

- The MAS plays a central orchestrating role across the AI agent layers and ADs as follows:
  - Core Responsibilities:
    - Coordinates intent propagation down from Product → Service → Resource layers.
    - Enables policy reconciliation, e.g., ensuring service objectives are consistent across services and resource constraints.
    - Triggers negotiation loops when intent conflicts arise between domains or layers.
    - Supervises execution dependencies (e.g., one service requires another to complete first).
    - May use game-theoretic or rule-based logic for distributed decision-making across domains.

© TM Forum 2025. All Rights Reserved.

4.3.1.1. MAS-Based Intent Management Across Product, Service, and Resource Domains

- **PHASE 1: Intent Declaration (Product Domain)**
  - Step 1: Intent Owner Agent
    - Captures high-level business requirements (e.g., customer SLA, service reliability).
    - Expresses the "what" — without specifying the "how".
  - Step 2: Intent Translator Agent
    - Converts declarative business goals into machine-readable structured intent.
    - These include QoS expectations, delay thresholds, or business policies.
    - Output is sent downward to the Service Domain.
- Note: This layer does not perform operational decisions — it delegates.
- **PHASE 2 – Intent Realization (Service Domain)**
  - Step 3: Intent Handler Agent
    - Receives translated goals from Product AD.
    - Converts them into service-specific actions or policies (e.g., provision 5G slice, reserve compute resources).
    - Coordinates with service orchestration systems.
  - Step 4: Intent Monitoring Agent
    - Actively tracks whether service goals are being met (e.g., is the latency < threshold?).
    - Identifies deviations or potential delay violations.
    - Sends feedback upward to Product Domain for possible escalation.
  - Step 5: Fault Recovery Agent (Service AD)
    - Detects anomalies in service logic, like broken dependencies or failed provisioning.
    - May attempt logical recovery, retries, or rerouting requests to alternate services.
  - Step 6: Multi-Agent Coordinator (Service Layer)
    - Orchestrates policy/intent negotiation when goals across domains conflict (e.g., competing KPIs measures).
    - Bridges Product-Service-Resource interactions to align service delivery with goals.
- PHASE 3 – Intent Enforcement (Resource Domain)
  - Step 7: Intent Handler Agent (Resource AD)
    - Applies service-derived rules into network/device configurations.

© TM Forum 2025. All Rights Reserved.

        - Example: Activates bandwidth reservation, assigns VLAN tags, provisions edge nodes.
    - Step 8: Intent Monitoring Agent (Resource AD)
    - Monitors real-time network telemetry (e.g., drop rate, jitter, throughput).
    - Detects deviations from intended performance goals.
- Step 9: Fault Recovery Agent (Resource AD)
    - Triggers self-healing procedures (e.g., reroute traffic, reset links).
    - Sends observed deviations back to Service Domain (e.g., low bandwidth detected).

Feedback Loops and Coordination Flows

- Northbound Feedback Arrows (↑):
    - Resource AD → Service AD → Product AD
    - Used when performance drops or failures occur.
- Southbound Intent Flow (↓):
    - Product AD → Service AD → Resource AD
    - Used for goal propagation and configuration delegation.
- Cross-Layer Coordination (↔):
    - Handled by the Multi-Agent Coordinator, ensuring all agents remain aligned with global objectives, KPIs, and operational states.

SHACL NodeShape Validation for AIAgent

For an AIAgent within a MAS, it's important to validate the data that describes the agent to ensure it:

- Complies with predefined structures and constraints: This helps ensure that the agent behaves predictably within the system.
- Avoids incorrect interactions: For example, preventing an IntentExpressAgent from being connected to an invalid IntentMonitoringAgent.
- Provides confidence in agent interactions: Ensuring that agents can interact safely without violating data constraints helps guarantee that the MAS works efficiently.

Use Case - Consider a situation where you have multiple AI agents working in a telecom network system. Each agent needs to:

- Handle specific tasks (e.g., intent expression, monitoring, negotiation).
- Interact with other agents to complete tasks.

SHACL NodeShape validation:

- You can ensure that all agents are correctly defined with proper properties (e.g., type, status, capacity).

© TM Forum 2025. All Rights Reserved.

- You can enforce that agents are linked properly (e.g., IntentNegotiationAgent must be linked to a valid IntentMonitoringAgent).
- You can validate that all properties adhere to the expected types and constraints before allowing the agent to interact with others in the system.

4.3.2. MAS Lifecycle Coordination

Figure 3 portrays how multiple AI agents coordinate across intent lifecycle phases—Discovery, Negotiation, and Fulfillment—to manage and operationalize service intents in an autonomous, MAS-driven environment.

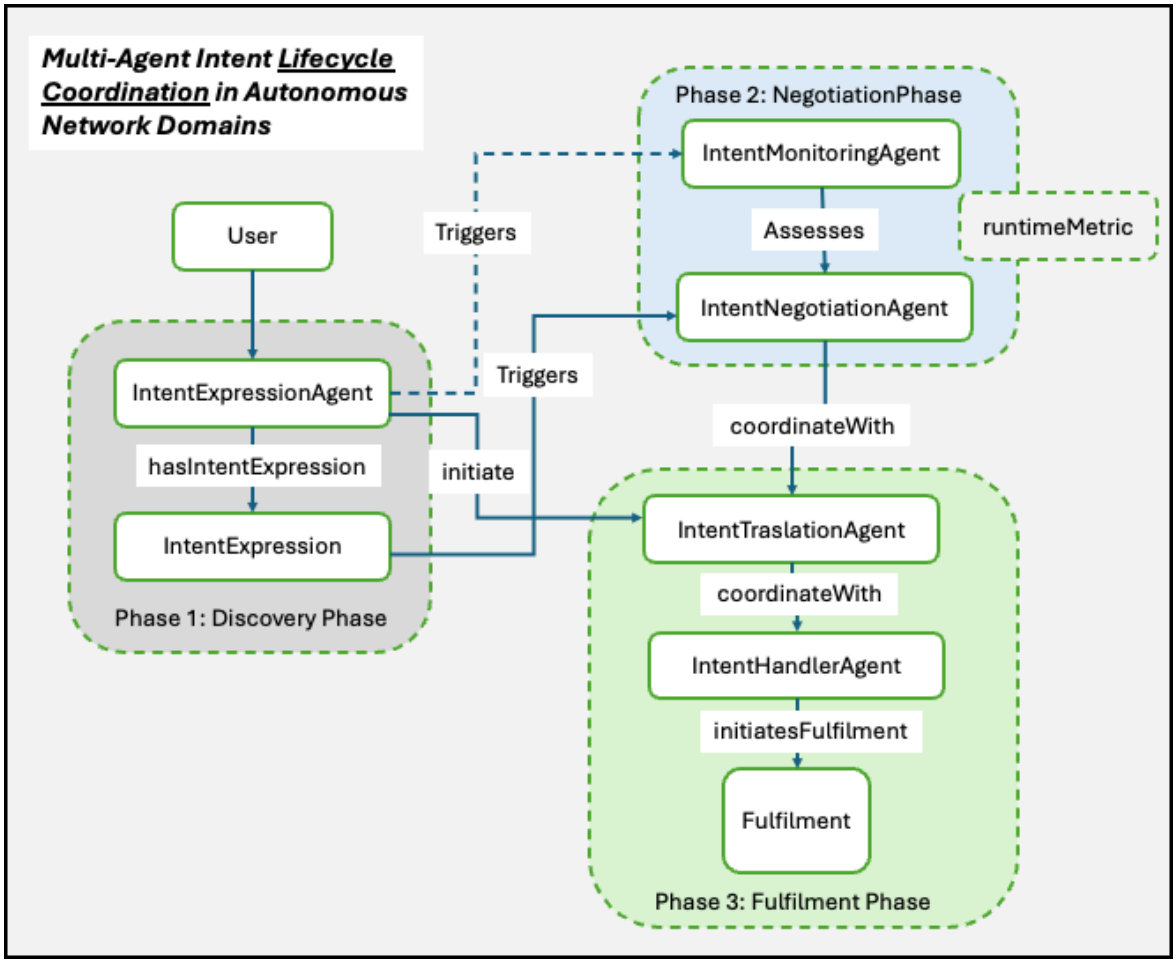

Figure 3: Multi-Agent Intent Lifecycle Coordination in Autonomous Network Domains

Figure 3 presents active AI agents that perform actions, hold semantic relations, or drive behavior such as.

- IntentExpressionAgent
- IntentMonitoringAgent
- IntentNegotiationAgent
- IntentTranslationAgent
- IntentHandlerAgent

- These are classes or instances of AIAgent of the ontology

© TM Forum 2025. All Rights Reserved.

- They can have properties like hasIntentExpression, coordinatesWith, delegatesTo
- These agents are involved in reasoning, transformation, or monitoring of intents

The dashed boxes Figure 4 represent intent lifecycle phases — abstract process stages, not entities such as:

- Discovery Phase
- Negotiation Phase
- Fulfillment Phase

These are not agents but are modeled as ex:IntentLifecyclePhase (an OWL Class) and used in object property ranges like initiatesPhase, transitionsTo. They structure the flow and contextualize actions of agents. And they help describe when agents operate in the multi-phase orchestration.

The Discovery Phase:

- Start the intent lifecycle
- The intent becomes semantically formalized (hasIntentExpression)
- This trigger point for IntentMonitoringAgent to start validation

User

↓

IntentExpressionAgent

├── hasIntentExpression → IntentExpression

├── initiatesPhase → DiscoveryPhase (dashed box)

└── triggers → IntentMonitoringAgent

PHASE 1: Discovery Phase

- Initiated by: IntentExpressAgent
- Action
  - Receives a user/customer intent input
  - Links the intent to an internal semantic object: IntentExpression
- Ontology Relation
  - IntentExpressAgent → hasIntentExpression → IntentExpression
- Next Step
  - Triggers the IntentMonitoringAgent to validate or monitor the intent (proactive validation)

© TM Forum 2025. All Rights Reserved.

PHASE 2: Negotiation Phase

- Transition: DiscoveryPhase → NegotiationPhase
- Agent Involved: IntentNegotiationAgent
- Action
  - Coordinates with the IntentExpressAgent to negotiate feasibility, policy, or KPI-related tradeoffs
  - Works with IntentHandlerAgent to assess implementation viability
  - Collects feedback through
    - coordinatesWith
    - delegatesTo
    - hasRuntimeMetric
- Purpose
  - Ensure that the intent is viable, compliant, and translatable into operational service terms

PHASE 3: Fulfillment Phase

- Initiated by: IntentTranslationAgent
- Action
  - Converts intent semantics into actionable workflows or service graphs
  - Delegates downstream to the IntentHandlerAgent
- Ontology Relation
  - IntentTranslationAgent → coordinatesWith → IntentHandlerAgent
  - IntentHandlerAgent → initiatesPhase → FulfillmentPhase

4.3.2.1. Intent Monitoring and Resolution

- Agent: IntentMonitoringAgent
- Action
  - Monitors IntentExpression compliance with the operational outcome
  - Resolves or triggers re-negotiation if deviation detected
- Relation
  - IntentMonitoringAgent → resolvesIntent → IntentExpression
  - IntentMonitoringAgent → resolvesIntent → RuntimeMetric

Table 11: Agent Collaboration

Agent Role	Core Function	Key Relations
IntentExpressAgent	Captures user intent	hasIntentExpression
IntentNegotiationAgent	Validates and negotiates feasibility	coordinatesWith, delegatesTo, hasRuntimeMetric
IntentTranslationAgent	Translates into executable logic	initiatesPhase, coordinatesWith
IntentHandlerAgent	Deploys or modifies service behavior	initiatesPhase, coordinatesWith

© TM Forum 2025. All Rights Reserved.

Agent Role	Core Function	Key Relations
IntentMonitoringAgent	Observes fulfillment and compliance Captures performance/state outcomes	resolvesIntent

4.3.2.2. MAS Lifecycle Coordination - Ontology

The ontology as shown in Figure 4 (enlarged in Figure 5) and the turtle file in Table 5 includes:

- IntentLifecyclePhase class and instances: DiscoveryPhase, NegotiationPhase, FulfillmentPhase
- MAS coordination properties: coordinatesWith, delegatesTo, initiatesPhase, resolvesIntent
- OWL restrictions for IntentExpression and RuntimeMetric
- SHACL NodeShape validation for AIAgent
- Programmatic discoverability via rdfs:seeAlso

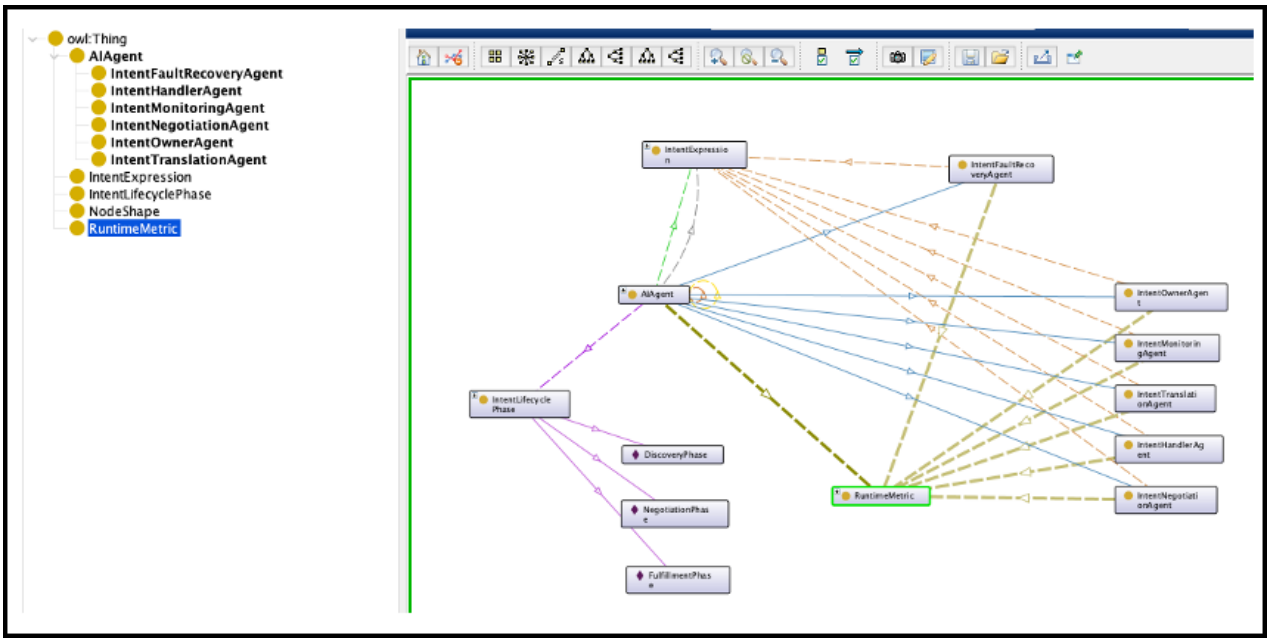

Figure 4: MAS Protege Class Hierarchy (see enlarged Figure 5 in Figure 6)

© TM Forum 2025. All Rights Reserved.

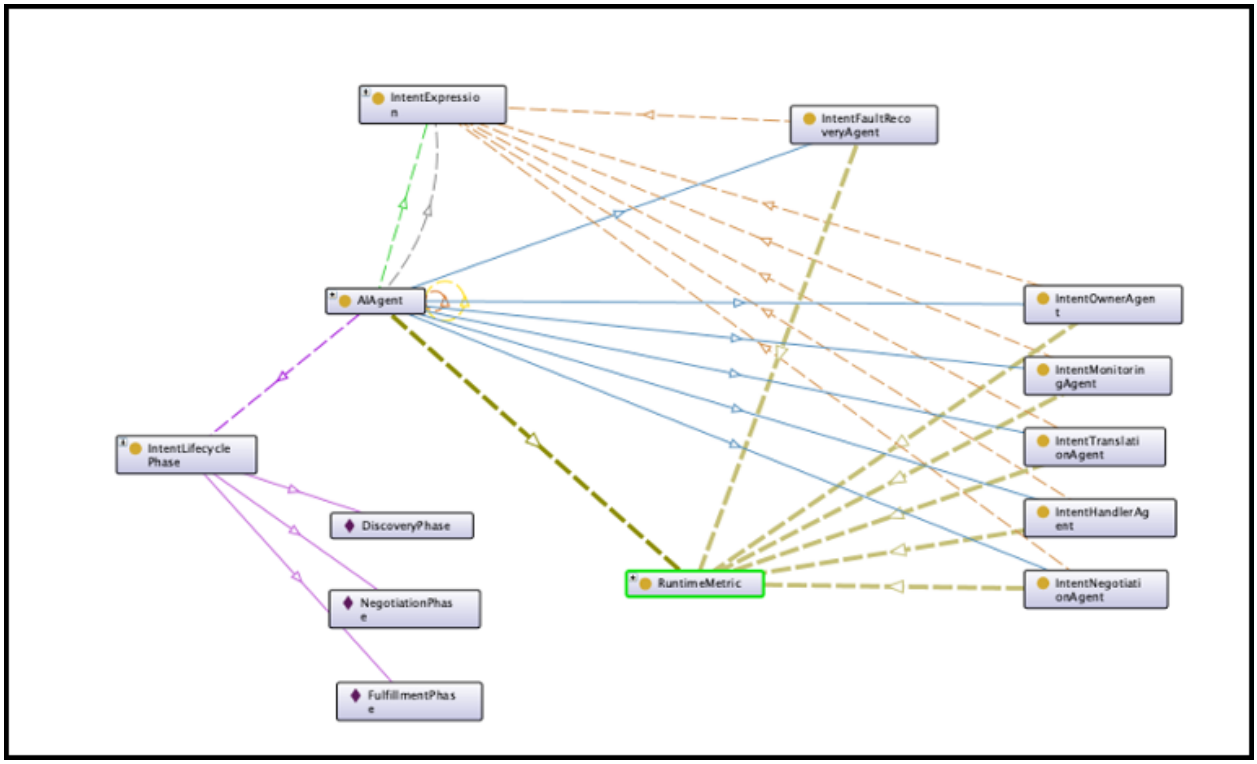

Figure 5: Figure 4 Enlarged for Clarity

Table 12: MAS Class Hierarchy Turtle File

MAS Class Hierarchy Turtle File
@prefix ex: <http://example.org/ontology#> . @prefix ns1: <http://www.w3.org/2002/07/owl#> . @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> . @prefix sh: <http://www.w3.org/ns/shacl#> . @prefix xsd: <http://www.w3.org/2001/XMLSchema#> . ex:DiscoveryPhase a ex:IntentLifecyclePhase . ex:FulfillmentPhase a ex:IntentLifecyclePhase . ex:IntentFaultRecoveryAgent a ns1:Class ; rdfs:subClassOf [a ns1:Restriction ; ns1:onProperty ex:hasRuntimeMetric ; ns1:someValuesFrom ex:RuntimeMetric], [a ns1:Restriction ; ns1:onProperty ex:hasIntentExpression ; ns1:someValuesFrom ex:IntentExpression], ex:AIAgent . ex:IntentHandlerAgent a ns1:Class ; rdfs:subClassOf [a ns1:Restriction ; ns1:onProperty ex:hasIntentExpression ; ns1:someValuesFrom ex:IntentExpression], [a ns1:Restriction ; ns1:onProperty ex:hasRuntimeMetric ; ns1:someValuesFrom ex:RuntimeMetric], ex:AIAgent . ex:IntentMonitoringAgent a ns1:Class ; rdfs:subClassOf [a ns1:Restriction ; ns1:onProperty ex:hasRuntimeMetric ; ns1:someValuesFrom ex:RuntimeMetric], [a ns1:Restriction ; ns1:onProperty ex:hasIntentExpression ;

© TM Forum 2025. All Rights Reserved.

```
        ns1:someValuesFrom ex:IntentExpression ],
      ex:AIAgent .

ex:IntentNegotiationAgent a ns1:Class ;
  rdfs:subClassOf [ a ns1:Restriction ;
        ns1:onProperty ex:hasIntentExpression ;
        ns1:someValuesFrom ex:IntentExpression ],
      [ a ns1:Restriction ;
        ns1:onProperty ex:hasRuntimeMetric ;
        ns1:someValuesFrom ex:RuntimeMetric ],
      ex:AIAgent .

ex:IntentOwnerAgent a ns1:Class ;
  rdfs:subClassOf [ a ns1:Restriction ;
        ns1:onProperty ex:hasRuntimeMetric ;
        ns1:someValuesFrom ex:RuntimeMetric ],
      [ a ns1:Restriction ;
        ns1:onProperty ex:hasIntentExpression ;
        ns1:someValuesFrom ex:IntentExpression ],
      ex:AIAgent .

ex:IntentTranslationAgent a ns1:Class ;
  rdfs:subClassOf [ a ns1:Restriction ;
        ns1:onProperty ex:hasIntentExpression ;
        ns1:someValuesFrom ex:IntentExpression ],
      [ a ns1:Restriction ;
        ns1:onProperty ex:hasRuntimeMetric ;
        ns1:someValuesFrom ex:RuntimeMetric ],
      ex:AIAgent .

ex:NegotiationPhase a ex:IntentLifecyclePhase .

ex:coordinatesWith a ns1:ObjectProperty ;
  rdfs:domain ex:AIAgent ;
  rdfs:range ex:AIAgent .

ex:delegatesTo a ns1:ObjectProperty ;
  rdfs:domain ex:AIAgent ;
  rdfs:range ex:AIAgent .

ex:initiatesPhase a ns1:ObjectProperty ;
  rdfs:domain ex:AIAgent ;
  rdfs:range ex:IntentLifecyclePhase .

ex:resolvesIntent a ns1:ObjectProperty ;
  rdfs:domain ex:AIAgent ;
  rdfs:range ex:IntentExpression .

ex:AIAgentShape a sh:NodeShape ;
  sh:property [ sh:class ex:IntentExpression ;
        sh:minCount 1 ;
        sh:path ex:hasIntentExpression ],
      [ sh:class ex:RuntimeMetric ;
        sh:minCount 1 ;
        sh:path ex:hasRuntimeMetric ] ;
  sh:targetClass ex:AIAgent .

ex:IntentLifecyclePhase a ns1:Class .

ex:hasIntentExpression a ns1:ObjectProperty ;
  rdfs:domain ex:AIAgent ;
  rdfs:range ex:IntentExpression .
```

© TM Forum 2025. All Rights Reserved.

```
ex:hasRuntimeMetric a ns1:ObjectProperty ;
   rdfs:domain ex:AIAgent ;
   rdfs:range ex:RuntimeMetric .

ex:RuntimeMetric a ns1:Class .

ex:IntentExpression a ns1:Class .

ex:AIAgent a ns1:Class ;
   rdfs:seeAlso ex:AIAgentShape .
```

© TM Forum 2025. All Rights Reserved.

5. AI Agent MAS and IMF

IMF agents facilitate intent lifecycle management, ensuring translation, execution, and monitoring.

I. AI Agent and Network & Resource Management

- Responsible for real-time network optimization, fault recovery, security, and SLA enforcement.

AI Agent Classification Based on Decision-Making Autonomy

- Reactive Agents
  - Static Policy-Based
- Deliberative Agents
  - Goal-Oriented AI Agents
- Learning Agents
  - Reinforcement Learning for Adaptation
- Hybrid Agents
  - Combining Rules + AI Learning

III. Interaction Model for MAS

- Intra-Domain Coordination
  - AI agents collaborate within a single domain.
- Inter-Domain Coordination
  - Cross-domain intent negotiation among IMFs

5.1. AI Agent and MAS Intent-Base Framework for AN Service Management

AI agent MAS operate at multiple layers with clear coordination & decision-making strategies

- **Product-Level AI Agents**
  - Define CFS intents and business-driven objectives.
- **Service-Level AI Agents**
  - Convert business intent into service-level policies.
- **Resource-Level AI Agents**
  - Manage **infrastructure resources** for intent fulfillment.

Table 14 and Table 15 summarizes the AI Agents MAS Functions and Characteristics for an Applications Domain.

Table 13: AI Agent MAS Use Cases and Applications Domain

Agent Name	Function	Example Use Case	Applicable Domain
Network Optimization Agent	Allocates resources dynamically for efficiency.	Adjusts routing paths to avoid congestion.	**Service & Resource-Level AD**

© TM Forum 2025. All Rights Reserved.

Agent Name	Function	Example Use Case	Applicable Domain
Fault Detection & Recovery Agent	Identifies faults and applies self-healing mechanisms.	Detects node or link failure and triggers the appropriate mitigation: MPLS Fast Reroute (< 50 ms) where protected paths exist, or CPE dual-homing / graceful service degradation where alternate routes are unavailable	**Service & Resource-Level AD**
Security Enforcement Agent	Implements and enforces security policies.	Correlates multi-layer telemetry to detect anomalous behavior (e.g., Volt Typhoon lateral movement) and enforces micro-segmentation / adaptive ACLs or slice isolation across domains	**Service & Resource-Level AD**
Service Quality Agent	Ensures SLA and QoS compliance through proactive adjustments.	Adjusts resource allocations to meet latency guarantees.	**Service-Level AD**

Table 14: AI Agent Types Characteristics Application Domain

Type of Agent	Characteristics	Example Use Case	Applicable Domain
Reactive Agents (Rule-Based)	Respond to environmental changes using predefined rules.	Firewall Agent blocks traffic based on static policies.	**Service & Resource-Level AD**
Deliberative (Goal-Driven) Agents	Perform reasoning based on goals and an internal state.	Network Slicing Agent reallocates resources dynamically.	**Product & Service-Level AD**
Learning (AI-Based) Agents	Use ML, RL, DL to improve performance over time.	RL-based Traffic Prediction Agent optimizes routing.	**Service & Resource-Level AD**
Hybrid Agents	Combine rule-based decision-making with ML for adaptive behavior.	Self-Optimizing AI Agent adjusts configurations based on real-time network conditions.	**Service & Resource-Level AD**

© TM Forum 2025. All Rights Reserved.

5.2. Knowledge Graph and Knowledge Base Representation of Intent Management State

Knowledge Graph (KG) is a structured data model that consists of:

- **Nodes (Entities)** - Represent AI agents, intent objects, network elements, and service states.
- **Edges (Relationships)** - Define interactions, dependencies, and state changes among entities.
- **Attributes (Properties)** - Capture metrics such as compliance status, optimization actions, and performance indicators.

An intent-based AI agent report leverages this structure to dynamically **model and** track the real-time state of an autonomous network.

Unlike a complete network graph, which models the entire system, an intent report is a partial knowledge graph focused on:

- The relevant subset of the network directly related to the intent being monitored.
- The current operational state at a specific observation point (e.g., timestamp or monitoring interval).
- Specific agents, services, and resources that directly impact—or are impacted by—the declared intent.

To ensure that this partial graph remains valid and usable across domains, it can be validated using SHACL (Shapes Constraint Language)**.**

- A SHACL node or property shape defines the expected structure of the instance data (e.g., presence of a compliance flag, associated metrics, and the temporal window). This approach supports intent report generation as a conformance check against expected knowledge graph patterns and may represent a practical enhancement aligned with initiatives such as BBF and IETF NMOP.

- TMF TR290B [37] can provide structural guidance for representing intent reports as partial graphs. These align with SHACL-conformant A-Box structures where compliance and deviation reports are extracted from runtime telemetry.

In practice, an intent report can be produced—or validated—by applying a SHACL node/property shape to the knowledge graph. The shape describes the expected structure (e.g., compliance flag, metric set, temporal window) and ensures the partial graph conforms before it is published or consumed.

© TM Forum 2025. All Rights Reserved.

For example, an Intent Monitoring Agent may generate a report showing:

- Compliance status - Whether the service-level agreement (SLA) or policy requirements are being met.
- Observed values - Metrics such as bandwidth, latency, or fault detection.
- Affected entities - Devices, links, or services impacted by a change in intent.
- Temporal validity - The timestamp or time window during which the report is applicable.

5.3. AI Agent Roles in Generating and Updating Knowledge Graph

In this section, we focus on how AI agents interact with the knowledge graph at the instance level (i.e., the A-Box, see Appendix D), where they dynamically generate and update data reflecting current intent, service, and network conditions. The underlying ontology (i.e., the T-Box, see Appendix D)—which defines classes, relationships, and constraints—is governed through a separate process to ensure stability and semantic integrity.

The knowledge graph consists of a governed ontology layer (T-Box), and a dynamic instance layer (A-Box), see Appendix D. AI agents only add or update A-Box triples at run-time; any change to the ontology itself follows the standard governance workflow.

In an intent-based service management system, multiple AI agents interact to populate and update instance-level data (A-Box) in the intent knowledge graph. The structural schema and domain concepts (T-Box), which define classes and relationships, are separately designed and governed by intent modeling teams or domain experts. The AI agents do not autonomously create new ontologies but rely on existing, version-controlled models—often mapped from heterogeneous sources such as YANG or OpenAPI definitions using tools like SHACL or R2RML.

AI agents that express or translate intent (e.g., Intent Owner, Intent Handler) follow modeling conventions from TR290A [36], using goal-oriented constructs, priorities, and declared scope as described in TMF Intent Expression models.

- Intent Owner Agent
    - Defines and updates intent instance data in the knowledge graph (A-Box), such as declared objectives, operational parameters, and compliance goals.
    - Ensures that all intent instances align with business policies, regulatory constraints, and operational capabilities.
    - Note: Ontology entities (T-Box)—which define the schema, classes, and relationships used by all agents—are not modified at runtime. Instead, they are maintained under a governed version-controlled process. AI agents consume the ontology to validate and structure instance updates but do not alter the ontology directly.

© TM Forum 2025. All Rights Reserved.

  - Caution: Modifying ontology-level structures (T-Box) at runtime can disrupt existing triple store consistency and impair inferencing performance. For that reason, the document distinguishes between runtime instance updates (A-Box, handled by AI agents) and schema evolution (T-Box, governed by design-time processes).
- Intent Handler Agent
  - Maps intent objectives (defined in A-Box) to actionable network configurations or policies, translating declared goals into deployable service and resource actions.
  - Tracks whether the current network state satisfies the declared intent by querying relevant telemetry and state metrics.
  - Note: These mappings are instance-level data operations (A-Box). No ontology (T-Box) constructs are modified; the agent uses existing classes and properties from the ontology to validate and contextualize its actions.

- Intent Monitoring Agent

  - Continuously queries and updates the knowledge graph.

  - Identifies compliance deviations and generates reports based on real-time data.

- Intent Negotiation Agent

  - Facilitates intent coordination across different ADs.

  - Updates the knowledge graph based on inter-domain agreements and policy modifications.

- Note: Ontology Integrity: Because heterogeneous YANG models differ across vendors, new resource concepts must first be mapped (e.g., via SHACL/R2RML) and reviewed before entering the T-Box. This avoids breaking existing triples or degrading query performance (see Table 16)."

Table 15: AI Agent Triple Store Activity Summary

AI Agent Role	Primary Function	Triple Store Activity
Intent Owner Agent	Defines and updates intent-related instances in the graph; aligns with business policies	A-Box (instance data); T-Box updates are governed separately
Intent Handler Agent	Maps intent objectives to actionable policies; checks compliance with network state	A-Box only (instance-level mappings, no ontology changes)
Intent Monitoring Agent	Continuously queries and updates real-time intent compliance data	A-Box only (telemetry and compliance observations)

© TM Forum 2025. All Rights Reserved.

AI Agent Role	Primary Function	Triple Store Activity
Intent Negotiation Agent	Coordinates and updates inter-domain intent agreements and policy changes	A-Box (cross-domain policy instance)

5.4. Semantic Nature of Intent Expressions and Their Operational Realization

Intent expressions are declarative semantic constructs—they describe what the desired outcomes are, not how to achieve them.

Intent expressions may contain metrics (e.g., latency < 50ms, throughput > 10Gbps) or thresholds that describe target states, service-level objectives, or operational goals. These expressions are consumed by policy engines or analytics modules that perform the necessary calculations and reasoning to derive enforcement actions.

5.5. Key Roles in the Intent Pipeline

Figure 6 shows the semantic representation of intent, which remains technology-agnostic, while the execution logic resides **in** programmable modules tuned to the operator's architecture, data sources, and performance goals.

This aligned with TMF IG1230, where the transformation of declarative business objectives into automated enforcement must traverse multiple AI agent layers and data pipelines.

- Intent Expression Layer: Defines the "what"—e.g., latency should be below **X** ms, packet loss below **Y%**, bandwidth utilization above **Z%**.
- Analytics & Policy Engines: Interpret semantic expressions, perform threshold comparisons, statistical evaluations, predictive analysis, or conflict resolution to determine actionable decisions.
- Actuation Layer: Enforces changes through configuration updates, control plane signaling, or orchestration mechanisms.

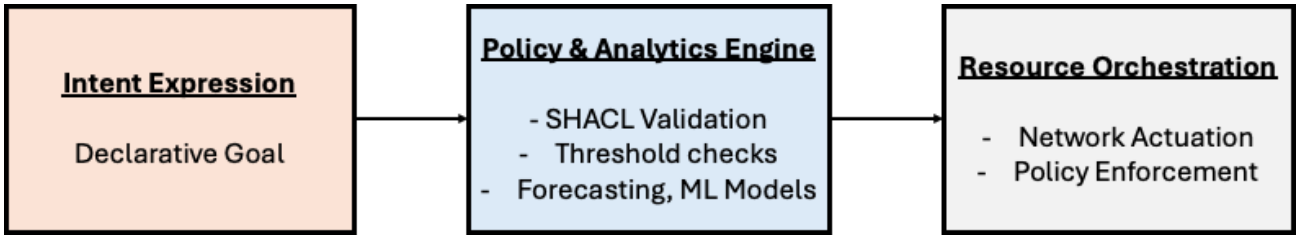

Figure 6: Intent-to-Action Pipeline

© TM Forum 2025. All Rights Reserved.

5.5.1. Bridging Declarative Intent and Operational Execution

The flow diagram Figure 7 illustrates the semantic pipeline for realizing intent in an autonomous service management system. It begins with Intent Expression, where user or system-defined objectives (e.g., performance, policy, reliability) are declared in a structured and technology-agnostic format. These are passed to the Semantic Parsing layer, which translates and maps the intent into formal representations aligned with domain ontologies and policy models. The Analytics Engine then interprets these parsed intents by evaluating current network state, telemetry, and performance metrics to determine whether the goals are achievable or need adaptation. Finally, the pipeline culminates in Action, where AI agents or orchestrators execute configuration changes, trigger workflows, or initiate optimization mechanisms based on the resolved intent. This flow underscores the decoupling of *declarative goals* from *operational execution*.

Figure 7: Semantic Execution of Network Intent

TMF TR290 [36] and its sub-models (A, B) formally define the expression and reporting structures that feed into the semantic parsing and analytics layers.

© TM Forum 2025. All Rights Reserved.

6. Intent-Based Service Management TMF APIs Ontology Support

The transition to AI-Agent MAS intent-based service management follows a structured, multi-steps evolution:

Step 1: AI agent MAS Intent-Based Assurance

- AI-powered intent monitoring agents
  - Track compliance and trigger corrective actions.
- TMF641 and TMF622 APIs
  - Support intent-based ordering and lifecycle management.
- AI-driven Intent Reports
  - Provide real-time compliance tracking.

Step 2: AI-Agent MAS Intent-Based Service Composition & Selection

- AI Agents autonomously select and compose RFSs.
- AI-driven orchestration determines the optimal RFS based on observed metrics.
- AI-enabled utility maximization ensures optimized decision-making.

Step 3: AI Agent MAS Intent Based adaptive Resource Management

- AI-based intent handlers dynamically reshape service instances.
- AI Agents identify optimization opportunities and execute corrective actions.
- AI-driven self-adaptive networks enhance autonomous operations.

Step 4: AI Agent MAS Intent Based Service Management

AI agent MAS into intent-based service management enables:

- Automation resilience and efficiency

Figure 8 shows AI-Agent API instances in autonomous domain. It consists of the API class and API instance in association with AI Agent class and autonomous domain class.

© TM Forum 2025. All Rights Reserved.

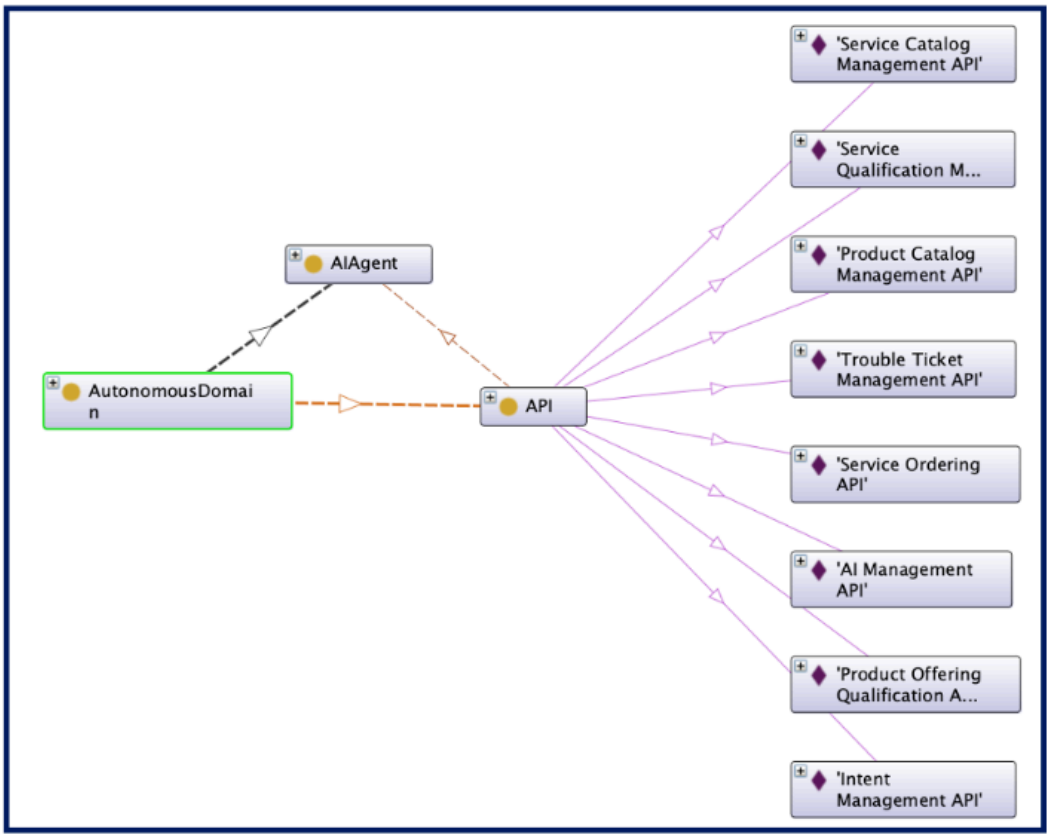

Figure 8: Protege API's instances in ADs

6.1. AI Agents MAS Intent based Ontology Support APIs

Intent APIs TMF921 allows seamless communication between Intent Owner Agents and Intent Handler Agents. API suites integrate AI-driven intent-based management into existing service and product management workflows. The following API enhancements support AI-driven service orchestration:

- Intent Specification in Service Catalogs
  - AI Agents align with TMF620 (Product Catalog Management) and TMF633 (Service Catalog Management) to structure and register intent-related metadata.
- Intent Qualification & Ordering
  - AI-enabled service intent validation is integrated into TMF641 (Service Ordering API) and TMF679 (Product Offering Qualification API) to ensure feasibility and compliance.
- Intent Lifecycle Management
  - AI-driven Intent Reports are generated through APIs like TMF645 (Service Qualification API) to track the compliance of services against defined intent goals.
- AI-Driven Feedback & Optimization
  - Intent Monitoring Agents provide feedback loops using TMF921, enabling real-time assessment and optimization.
- Managing AI models using AI Management API

© TM Forum 2025. All Rights Reserved.

- TMF915 provides a **standardized** approach to managing AI models, AI contracts, and AI-related resources in enterprise environment**.** It facilitates AI model lifecycle management, contract enforcement, monitoring, and event-driven interactions.

6.1.1. TMF620 API Product Catalog Ontology Support

The TMF620 Product Catalog Management API allows CSPs to manage product offerings, specifications, and lifecycle processes. TMF620 API is used at the Product Level AD, but it has implications for the Service Level AD and Resource Level AD. In the AD Ontology, TMF620 is associated to:

- ProductLevelAD to manage product catalogs and offerings.
- ServiceLevelAD to ensure service components align with cataloged offerings.
- AI Agents and IMF to automate the catalog update and intent negotiation.

TMF 620 Key Functionalities

- Lifecycle Management
  - Managing the entire lifecycle of product offerings (creation, update, deletion).
  - Ensuring products are published, launched, and retired correctly,
- Catalog Lookup & Querying
  - AI agents can query the catalog to determine available services, for example, a user requests an enterprise VPN service, and an AI agent fetches valid product offerings.
- Customization & Versioning
  - AI agents can generate product variants dynamically, for instance, a customer selects a cloud computing service, and AI suggests custom configurations.
- Order & Qualification Support
  - Works with TMF679 (Product Offering Qualification API) to determine if a product is eligible for ordering.
- AI enhances TMF620 by automating and optimizing product catalog updates, pricing strategies, and lifecycle transitions.
- AI Agent Catalog Management
  - Uses ML to predict product demand.
  - Suggests new catalog updates based on customer trends.
- AI Agent Intent-Based
  - Works with IMF to automate catalog updates based on intents.
  - Ensures new services are cataloged dynamically.
- Pricing Optimization AI Agent
- Uses RL to suggest dynamic pricing, for instance discounts based on competitor pricing.

© TM Forum 2025. All Rights Reserved.

MAS Enhancements

- Autonomous AI Coordination
  - AI agents collaborate across different layers (Product → Service → Resource), for instance, a Service AI Agent ensures product catalog updates trigger network configuration changes.
- Real-Time Catalog Updates
  - AI Agents auto-detect network changes and update catalog entries, for instance, edge computing services update dynamically based on network latency.
- Cross-Domain AI Negotiation
  - Uses Game Theory & Multi-Agent Negotiation to balance resources, for instance, Cloud & Network AI Agents collaborate to **optimize** service delivery.

6.1.2. TMF633 Service Catalog Management API Ontology Usage

The TMF633 Service Catalog Management API enables the management of service catalog elements, including their lifecycle, and facilitates service catalog consultation during ordering and service fulfillment.

Ontology TMF633 API Usage Key Functionalities

- Lifecycle Management
  - Handles service catalog elements from creation to retirement.
- Service Specification Management
  - Defines and structures service offerings.
- Service Candidate & Service Category
  - Manages service offerings before they become available for ordering.
- Import/Export Capabilities
  - Synchronizes catalogs across different systems.
- Notification Events
  - Tracks changes in service catalog elements.

Mapping to Ontology: Service Catalog in the Autonomous Domain Model

- The TMF633 API fits within the Service Level AD and Resource Level AD.
- It links Service Catalogs to CFSs and RFSs.

Table 18 shows TMF 633 API mapping

Table 16: TMF633 API Mapping

Ontology Concept	TMF633 API Mapping
RFS & CFS Mapping	Service specifications define mappings between ServiceLevelAD & ResourceLevelAD
Service Candidate	ServiceCandidate entity
Service Catalog	ServiceCatalog resource
Service Category	ServiceCategory resource
Service Specification	ServiceSpecification resource

© TM Forum 2025. All Rights Reserved.

- Class
  - ServiceCatalogManagementAPI to represent TMF633.
- New Object Properties
  - usesServiceCatalogAPI to link ServiceLevelAD and **R**esourceLevelAD to ServiceCatalogManagementAPI.
  - hasServiceSpecification to associate Service Specification with AI Agents managing catalogs.

Table 18 shows AI agents' enhancement using TMF633 API.

Table 17: AI Agents enhancement using TMF633 API

AI Agent	Enhancement Using TMF633 API
ServiceCatalogAIAgent	Automates service catalog updates and synchronization
LifecycleAIAgent	Predicts and manages lifecycle transitions (e.g., retirement, upgrades)
IntentAIAgent	Aligns service catalog with intent-based networking
RecommendationAIAgent	Uses AI to suggest **new service offerings** based on demand
AutoDiscoveryAIAgent	Identifies and categorizes new service candidates dynamically

6.1.3. TMF679 Product Offering Qualification API Ontology Usage

TMF679 Product Offering Qualification API is part of the Pre-Ordering Management API Family, designed to:

- Check if a product offering is commercially eligible based on customer context.
- Query product offering availability based on product category or specification.

This API interacts with Catalog and Inventory systems to ensure consistent service qualification processes.

The API has two main components:

- CheckProductOfferingQualification – Requests eligibility verification of one or more product offerings.
- QueryProductOfferingQualification – Queries for product offerings based on predefined product categories or specifications.

Operations Supported

- Retrieve product offering qualification results.
- Partially update a product offering qualification.
- Create a new product offering qualification.
- Delete a product offering qualification (administrative purposes).
- Notify events related to product offering qualifications.

© TM Forum 2025. All Rights Reserved.

Lifecycle States

- In Progress
  - Sales team is processing the qualification request.
- Acknowledged
  - Request is received and validated.
- Terminated With Error
  - Qualification could not be completed.
- Done
  - Qualification process is complete.

TMF679 Ontology Integration

TMF679 is relevant for Product-Level Autonomous Domains (ProductLevelAD), which manages CFS.

Ontology Mapping

- TMF679 API should be associated with ProductLevelAD because:
  - Product Offering Qualification determines commercial eligibility.
  - It interacts with the Catalog Management API (TMF620) to ensure available offerings.
  - It supports decision-making at the product level before service orchestration.
- New Properties & Links in Ontology
  - API Class: ProductOfferingQualificationAPI
  - Relation to ProductLevelAD

AI Agent TMF679 API Usage

AI agents ADs can optimize product offering qualification by:

- Predicting Customer Demand:
  - AI models can analyze historical product requests to predict future customer demand. For example, AI can suggest alternate product offerings when certain products are unavailable.
- Dynamic Product Recommendations:
  - Multi-agent AI systems can personalize product offers based on user location, past purchases, and eligibility. For example: AI agents can suggest the best broadband package based on geographic location.
- Automated Decision-Making in Autonomous Domains:
  - AI Intent Management Functions (IMF) can automate qualification checks for product eligibility. For example, an AI agent in Product-Level AD could interact with Service-Level AD to determine if a service is available based on resource constraints.
- Closed-Loop Optimization in CFS and RFS:

© TM Forum 2025. All Rights Reserved.

  - AI agents can refine product offerings dynamically based on real-time service performance. For example, if a 5G package is unavailable in an area, AI can suggest alternative network solutions.
- Inter-Agent Negotiation for Product Qualification:
  - AI multi-agent systems can negotiate pricing, availability, and SLA requirements dynamically. For example, an AI-powered chatbot can directly interact with TMF679 API to suggest available offers.

6.1.4. TMF915 AI Management API Ontology Usage

- Standardized AI Model Management
  - Manages CSP and enterprises AI models in a structured and API-driven
  - Ensures compatibility across multiple vendors and AI systems.
- AI Contract Specification & Compliance
  - Monitors compliance with defined rules (e.g., performance benchmarks, SLAs).
- Supports AI Lifecycle Operations
  - AI model creation, versioning, modification, and retirement.
  - Support for AI model monitoring and notifications.
  - Automate AI lifecycle management
  - Reduce manual intervention in AI models; onboarding, updated, and retirement
- Event-Driven & Rule-Based AI Management
  - Triggers real-time alerts on AI model contract violations or issues.
  - Supports threshold rules to adjust AI models dynamically.
- Supports AI Governance & Compliance
  - Ensures AI models follow contractual performance agreements (e.g., accuracy, fairness, latency, e.g., service agreements, operational limits, etc.).
- Enhances AI Performance Monitoring
  - Tracks AI health, contract violations, and operational limits.
- Event-Driven AI Adaptation
  - Uses real-time alerts and rule-based enforcement to trigger AI retraining or adjustments.

Table 19 shows TMF915 API Common Operations

Table 18: TMF915 API Common Operations

Action	Endpoint	Description
Create AI model	POST /aiModel	Registers a new AI model.
Retrieve AI model	GET /aiModel/{id}	Fetches details of a specific AI model.
Update AI model	PATCH /aiModel/{id}	Modifies an existing AI model.
Delete AI model	DELETE /aiModel/{id}	Removes an AI model from inventory.
List AI contract	GET /aiContract	Fetches all AI contracts.

© TM Forum 2025. All Rights Reserved.

Action	Endpoint	Description
Monitor AI contract Violation	GET /aiContractViolation	Retrieves contract breaches.

AI Agents & MAS TMF915 API Usage

TMF915 API manages AI Agents and AI MAS. The API provides AI lifecycle management, contract enforcement, and monitoring capabilities, which are essential for autonomous AI agents operating in a telecom or enterprise environment. Table 20 shows TMF915 API Supports AI Agents & Multi-Agent Systems use case and endpoints.

Table 19: TMF915 Use cases and endpoints

Feature	AI Agent/MAS Use Case	Relevant TMF915 Endpoint
Register AI Agent Model	AI agents onboarded as managed entities.	POST /aiModel
Update AI Model for Retraining	Agents can adapt based on new data.	PATCH /aiModel/{id}
AI Model Governance & Monitoring	Tracks AI agent performance and compliance.	GET /aiContractViolation
AI-Agent Collaboration via Contracts	Ensures AI agent negotiations meet predefined SLAs.	POST /aiContract
Trigger Events for AI Agents	MAS-based event-driven execution.	GET /aiEvent/{id}

Scenario: A self-optimizing telecom network operates using multiple AI agents:

- AI Network Optimization Agent (adjusts network parameters).
- AI Traffic Prediction Agent (analyzes congestion patterns).
- AI Fault Detection Agent (detects service failures).

TMF915 API for AI Agent MAS

Standardized AI Governance

- AI models can be registered, monitored, and updated dynamically.
- Ensures AI agents operate within predefined SLA constraints.

Supports Adaptive AI-Agent Learning

- AI models can be retrieved and retrained on the fly (PATCH /aiModel, see Table 21).
- Multi-agent interactions can be optimized dynamically.

Event-Driven AI System

- Triggers notifications when AI agents exceed operational limits.
- Helps multi-agent systems self-adapt based on API event feedback.

© TM Forum 2025. All Rights Reserved.

Table 20: TMF915 Common API Operations

Action	Endpoint	Description
Create AI model	POST /aiModel	Registers a new AI model.
Retrieve AI model	GET /aiModel/{id}	Fetches details of a specific AI model.
Update AI model	PATCH /aiModel/{id}	Modifies an existing AI model.
Delete AI model	DELETE /aiModel/{id}	Removes an AI model from inventory.
List AI contract	GET /aiContract	Fetches all AI contracts.
Monitor AI contract Violation	GET /aiContractViolation	Retrieves contract breaches.

TMF915 API Across Operation Layers

TMF915 AI Management API can be used across all three layers: Product, Service, and Resource. However, how it is used differs at each layer, depending on the role of AI models and AI agents in the telecom ecosystem (see Table 22).

Product Layer: AI for Business & Customer-Facing Operations

- AI helps with product lifecycle automation (e.g., AI-driven pricing, customer analytics).
- TMF915 monitors AI compliance with business policies.

Service Layer: AI for Service Orchestration & Assurance

- AI-driven intent fulfillment & SLA monitoring.
- Ensures AI agents work together for end-to-end service quality.

TMF915 API Across Operation Layers

TMF915 AI Management API can be used across all three layers: Product, Service, and Resource. However, how it is used differs at each layer, depending on the role of AI models and AI agents in the telecom ecosystem.

Product Layer: AI for Business & Customer-Facing Operations

- AI helps with product lifecycle automation (e.g., AI-driven pricing, customer analytics).
- TMF915 monitors AI compliance with business policies.

Service Layer: AI for Service Orchestration & Assurance

- AI-driven intent fulfillment & SLA monitoring.
- Ensures AI agents work together for end-to-end service quality.

Resource Layer: AI for Network & Infrastructure Automation

- AI controls compute, storage, and network resources dynamically.
- TMF915 ensures self-healing, predictive maintenance, and adaptive AI models.

Mapping AI API to the Product, Service, and Resource Layers. Table 22 shows TMF915 use cases usage.

© TM Forum 2025. All Rights Reserved.

Table 21: TMF915 API Use Cases Usage

Layer	How TMF915 API is Used	Use Cases
Product Layer (Business Level AI Governance)	- AI models assist in business decision-making and customer interactions. - Ensures AI follows business-level compliance rules (e.g., SLA monitoring).	AI-powered product recommendations (e.g., "Recommend a 5G service based on customer profile").
Service Layer (Service Orchestration & AI SLAs)	- AI models manage service assurance, anomaly detection, and optimization. - AI contracts enforce QoS-based SLAs for AI-driven services. - Supports multi-agent AI collaboration.	Intent-driven AI orchestration (e.g., "Automatically optimize a service intent using AI agents").
Resource Layer (AI for Infrastructure & Automation)	- AI models control network, compute, and storage resources. - AI contracts ensure latency, energy, and cost-efficiency goals are met. - Supports self-healing AI-driven resource allocation.	Predictive AI-based network maintenance (e.g., "Detect and resolve faults before they impact customers").

6.1.5. AI Agent IMF APIs Integration

Customer request is received at the product layer IMF, which is decomposed to CFSs and RFS (aka Network Service), then it is passed to the Service layer IMF. These are different ADs and the relevant IMF in each domain at the boundary of these domains perform the integration between these two ADs. Intent based IMF enables the integration programmatically through open APIs. The model aligns with the CFS, RFS, and AD definitions. The model ensures (see Figure 9 Protégé AD hierarchy):

- CFS represents the commercial service abstraction that customers interact with.
- RFS represents the technical resource-bound service at the resource domain.
- ADs are structured around closed-loop self-management with decision autonomy.
- Intent propagates vertically and horizontally while maintaining clear service boundaries.

© TM Forum 2025. All Rights Reserved.

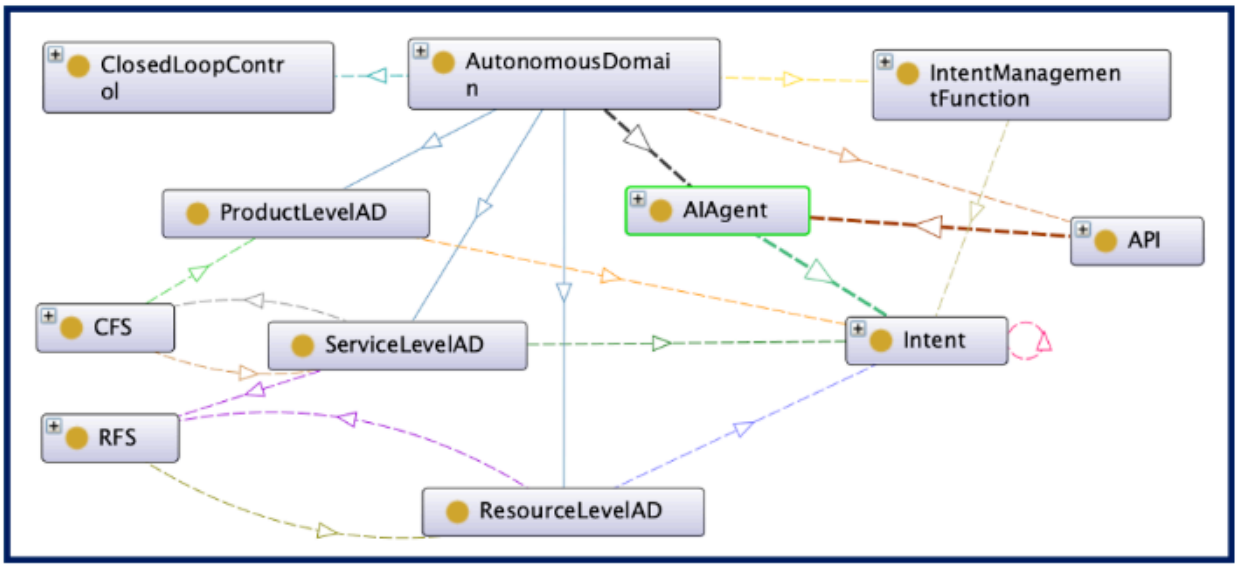

Figure 9: Protégé AD Hierarchy

© TM Forum 2025. All Rights Reserved.

7. AI Agent Intent-Based Ontology for Autonomous Domain Model

The ontology is used to model intent-driven autonomous domains in OWL-based reasoning systems. Protégé visualization is in Appendix A and the Turtle syntax is in Appendix B.

7.1. Ontology Representation Components

- Autonomous Domain (AD) is an Abstract Class with Product-Level AD, Service-Level AD, and Resource-Level AD as subclasses.
    - Product-Level AD :ProductLevelAD (Handles commercial intent, maps to CFS)
    - Service-Level AD :ServiceLevelAD (Translates CFS into Service Intent, maps to RFS)
    - Resource-Level AD :ResourceLevelAD (Implements RFS by orchestrating resources**)**

Table 23 shows autonomous domain ontology representation

Table 22: AD Ontology Representation

Description	Ontology Representation
Product-Level handles commercial intent and maps it to CFS.	:ProductLevelAD class, linked to CFS using :associatedWithProductAD.
Service-Level translates CFS into Service Intent before passing it down.	:ServiceLevelAD class, linked to CFS and RFS using :mapsToService and :mapsToResource.
Resource-Level executes RFS by orchestrating resources.	:ResourceLevelAD class, linked to RFS using :associatedWithResourceAD.

7.2. AI Agent Intent-Based Flows

Unlike AI Agents, Closed-Loop Control, and IMF, Intent is not a core operational component of an AD. Instead, Intent is a contextualized requirement that flows through different AD levels.

Intent is Assigned to Specific Levels in the Hierarchy

- Product-Level Intent (Handled by ProductLevelAD)

© TM Forum 2025. All Rights Reserved.

  - The customer expresses an intent, and the Product-Level AD translates it into CFS requirements.
  - Example: “I want a premium 5G service with ultra-low latency.”
- Service-Level Intent (Handled by ServiceLevelAD)
  - The Service-Level AD receives refined intent from the Product-Level AD and maps it to technical service functions.
  - Example: “This premium 5G service requires network slicing and traffic prioritization.”
- Resource-Level Intent (Handled by ResourceLevelAD)
  - The Resource-Level AD executes intent by assigning computing, network, and storage resources.
  - Example: “Allocate 100 MHz of spectrum in the 3.5 GHz band.”
- Intent is always context-specific and assigned to a specific AD level rather than to the general AutonomousDomain class.
- :IntentManagementFunction is linked to each AD (hasIMF).
- :AIAgent is associated with each AD (hasAgent).
- Each AD has intra-domain coordination via intraADNegotiation and inter-domain via interADNegotiation.
- Intent Management Function (IMF) and AI Agent representation within Each AD.
  - Each Product-Level AD, Service-Level AD, and Resource-Level AD has its own IMF through the has IMF property.
  - Each AD also has multiple AI Agents through the hasAgent property.
  - Links each AD to an IMF.

7.2.1. Ontology Intent Management Function (IMF) Properties

Every AD (Product, Service, Resource) has its own IMF

  - All IMFs function under the same governance principle
  - ServiceAD IMF provides the end-to-end management
- IMF is responsible for intent interpretation, execution, and negotiation.
- IMF is associated with AutonomousDomain
- IMF Coordinates AI Agents within AD
  - hasAgent property links AutonomousDomain to AIAgent.
- IMF Manages Inter-AD Coordination
  - interADNegotiation property links IntentManagementFunction to Negotiation.
- IMF Handles Intent Propagation
  - hasProductIntent, hasServiceIntent, hasResourceIntent properties link intents to corresponding ADs.
- IMF Handles Intent Optimization
  - optimizesIntent property links AIAgent to Intent.
- IMF Applies Optimization Function
  - hasUtility property links AIAgent to UtilityFunction.
- MAS Manages Multiple Agents per AD
  - AIAgent exists as a class and is correctly linked to AutonomousDomain via hasAgent.
- IMF-to-IMF Negotiation

© TM Forum 2025. All Rights Reserved.

    - interADNegotiation property links IntentManagementFunction instances across domains
- Intent Decomposition
    - hasSubIntent property links Intent objects to their decomposed sub-intents.

© TM Forum 2025. All Rights Reserved.

8. Appendix A: IMF for AD - AI Agent Protégé Ontology

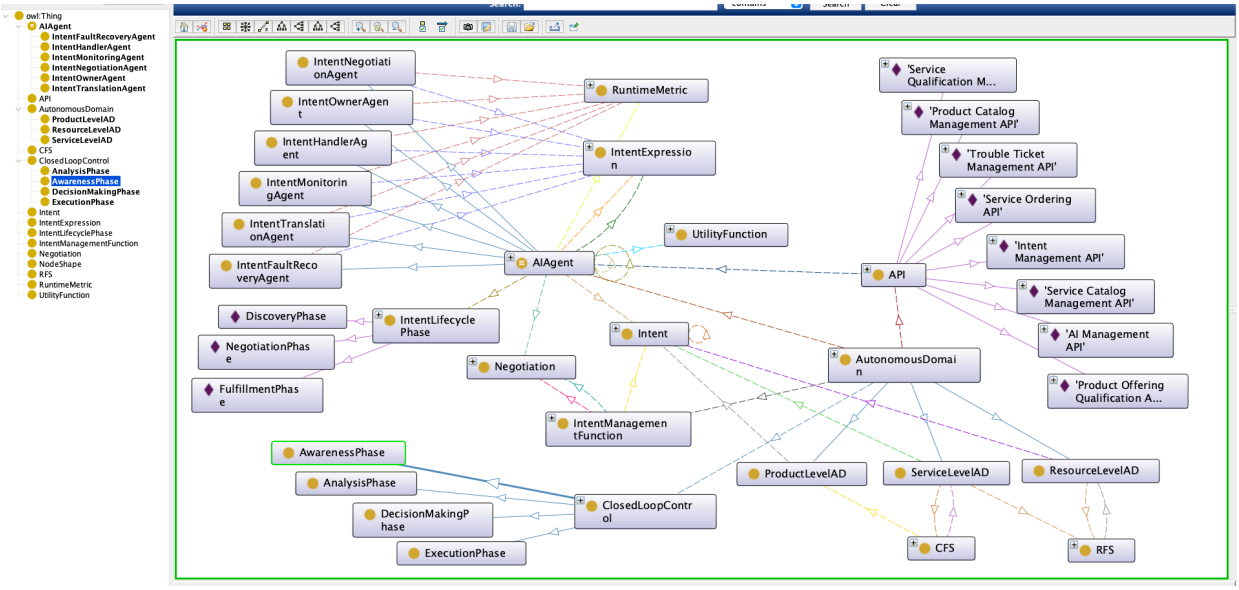

Figure 10: Protégé IMF for AD and AI Agent MAS Ontology

Figure 10 shows the complete IMF and APIs ontology interactions with all APIs instances, product, service and resource ADs, close loop control, intent-based AI Agents, intent lifecycle and its relevant instances, utility functions and the negotiation and IMF entities.

The ontology provides a formal, OWL-based vocabulary for intent-driven, AI-agent-orchestrated service management in autonomous telecom domains. It captures:

- Entities – AI agents, intents, autonomous domains (ADs), intent-management functions (IMFs), lifecycle phases, utility functions, metrics, negotiation sessions.
- Relationships – how intents flow down (product → service → resource), how feedback flows up, and how agents coordinate, negotiate, and optimise.
- Constraints & validation – SHACL node / property shapes ensure every instance conforms to required structure (e.g., each IntentMonitoringAgent must link to a RuntimeMetric).

Table 23: Ontology Prefix and Namespace

Prefix	Namespace	Notes
imac: *(default)*	http://example.org/intent#	Core intent & agent vocabulary
ad:	http://example.com/autonomous_domain_ontology#	AD hierarchy & domain metadata

© TM Forum 2025. All Rights Reserved.

Prefix	Namespace	Notes
ont:	http://example.org/ontology#	Utility, metric, supporting classes
standard	rdf, rdfs, owl, xsd, sh	RDF/S, OWL, XML schema, SHACL

Table 24: Core Class Hierarchy

Core class hierarchy
imac:AIAgent ├─ imac:IntentExpressionAgent ├─ imac:IntentMonitoringAgent ├─ imac:IntentNegotiationAgent ├─ imac:IntentTranslationAgent └─ imac:IntentHandlerAgent imac:Intent (abstract parent) ├─ imac:ProductIntent ├─ imac:ServiceIntent └─ imac:ResourceIntent imac:IntentLifecyclePhase ├─ imac:DiscoveryPhase ├─ imac:NegotiationPhase └─ imac:FulfillmentPhase ad:AutonomousDomain ├─ ad:ProductLevelAD ├─ ad:ServiceLevelAD └─ ad:ResourceLevelAD imac:IntentManagementFunction ont:UtilityFunction ont:RuntimeMetric

© TM Forum 2025. All Rights Reserved.

```
imac:Negotiationimac:AIAgent
  ├─ imac:IntentExpressionAgent
  ├─ imac:IntentMonitoringAgent
  ├─ imac:IntentNegotiationAgent
  ├─ imac:IntentTranslationAgent
  └─ imac:IntentHandlerAgent
imac:Intent            (abstract parent)
  ├─ imac:ProductIntent
  ├─ imac:ServiceIntent
  └─ imac:ResourceIntent
imac:IntentLifecyclePhase
  ├─ imac:DiscoveryPhase
  ├─ imac:NegotiationPhase
  └─ imac:FulfillmentPhase
ad:AutonomousDomain
  ├─ ad:ProductLevelAD
  ├─ ad:ServiceLevelAD
  └─ ad:ResourceLevelAD
imac:IntentManagementFunction
ont:UtilityFunction
ont:RuntimeMetric
imac:Negotiation
```

© TM Forum 2025. All Rights Reserved.

All AI-agent subclasses inherit from one unified imac:AIAgent class.

Table 25: Key object properties

Property	Domain → Range	Purpose
imac:hasIntentExpression	Agent → IntentExpression	links an intent producer to its declaration
imac:initiatesPhase	AIAgent → IntentLifecyclePhase	marks when an agent triggers Discovery, Negotiation, or Fulfillment
imac:coordinatesWith	AIAgent ↔ AIAgent	horizontal collaboration
imac:delegatesTo	AIAgent → AIAgent	vertical hand-off (e.g., Translator → Handler)
imac:resolvesIntent	IntentMonitoringAgent → Intent / RuntimeMetric	validation & compliance
imac:hasSubIntent	Intent → Intent	hierarchical decomposition (product → service → resource)
imac:optimizesIntent	AIAgent → Intent	links optimisation logic
ad:hasAgent	AutonomousDomain → AIAgent	places agents inside an AD
ad:hasIMF	AutonomousDomain → IntentManagementFunction	each AD's coordination loop
ad:interADNegotiation / ad:intraADNegotiation	IMF → Negotiation	cross-/intra-domain conflict resolution

8.1.1. SHACL validation layer

- sh:NodeShape targets ensure every AIAgent instance carries required capability links.
- Shapes for Intent objects check presence of priorities, target metrics, and temporal windows.
- Phase shapes verify that each Negotiation outcome points back to exactly one Intent expression.

8.1.2. Reasoning & usage scenarios

1. Lifecycle alignment – SWRL or SPARQL rules can infer that a proposed service intent must enter NegotiationPhase if hasConflict true.
2. Cross-domain queries – Ask "Which Resource-level intents roll up to product intent by following hasSubIntent*.
3. Compliance dashboards – A MonitoringAgent publishes RuntimeMetric instances; SHACL shapes flag non-conforming triples; dashboards read the graph for red/green status.

© TM Forum 2025. All Rights Reserved.

4. Automated negotiation – IntentNegotiationAgent instances consume Negotiation artefacts and output revised intents linked via resolvesIntent.
5. Closed-loop optimization – Utility functions stored as ont:UtilityFunction objects let agents rank alternative service variants.

8.1.3. Alignment with TM Forum references

- IMF mirrors TMF IG1253's Intent Management Function.
- Product/Service/Resource AD classes reflect TMF eTOM layered architecture.
- TMF 620 / 633 / 921 API mapping facilitated by rdfs:seeAlso links on classes like ProductIntent and IntentHandlerAgent.

8.2. Turtle File for MAS AI Agent Protégé Ontology

Table 26 provides turtle syntax for the MAS AI Agent Protégé ontology.

Table 26: Turtle syntax of the complete Ontology

© TM Forum 2025. All Rights Reserved.

```
@base <http://example.org/intent#> .
@prefix ad: <http://example.com/autonomous_domain_ontology#> .
@prefix ont: <http://example.org/ontology#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ad:AnalysisPhase a owl:Class ;
   rdfs:subClassOf ad:ClosedLoopControl .
ad:AutonomousDomainOntology a owl:Ontology .
ad:AwarenessPhase a owl:Class ;
   rdfs:subClassOf ad:ClosedLoopControl .
ad:CatalogManagementAIAgent a <http://example.org/AIAgent> ;
   rdfs:label "AI Agent for Catalog Updates" ;
   ad:optimizesIntent ad:ProductLevelAD .
ad:DecisionMakingPhase a owl:Class ;
   rdfs:subClassOf ad:ClosedLoopControl .
ad:ExecutionPhase a owl:Class ;
   rdfs:subClassOf ad:ClosedLoopControl .
ad:IntentBasedAIAgent a <http://example.org/AIAgent> ;
   rdfs:label "AI Agent for Intent-Driven Catalog Management" ;
   ad:optimizesIntent ad:ServiceLevelAD .
ad:PricingOptimizationAIAgent a <http://example.org/AIAgent> ;
   rdfs:label "AI Agent for Dynamic Pricing" ;
   ad:optimizesIntent ad:ProductLevelAD .
ad:associatedWithProductAD a owl:ObjectProperty ;
   rdfs:domain ad:CFS ;
   rdfs:range ad:ProductLevelAD .
ad:associatedWithResourceAD a owl:ObjectProperty ;
   rdfs:domain ad:RFS ;
   rdfs:range ad:ResourceLevelAD .
ad:associatedWithServiceAD a owl:ObjectProperty ;
   rdfs:domain ad:CFS ;
   rdfs:range ad:ServiceLevelAD .
ad:engagesInNegotiation a owl:ObjectProperty ;
   rdfs:domain ad:IntentManagementFunction ;
   rdfs:range ad:Negotiation .
ad:hasAgent a owl:ObjectProperty ;
   rdfs:domain ad:AutonomousDomain ;
   rdfs:range <http://example.org/AIAgent> .
ad:hasClosedLoop a owl:ObjectProperty ;
   rdfs:domain ad:AutonomousDomain ;
   rdfs:range ad:ClosedLoopControl .
ad:hasIMF a owl:ObjectProperty ;
   rdfs:domain ad:AutonomousDomain ;
   rdfs:range ad:IntentManagementFunction .
ad:hasProductIntent a owl:ObjectProperty ;
   rdfs:domain ad:ProductLevelAD ;
   rdfs:range ad:Intent .
ad:hasResourceIntent a owl:ObjectProperty ;
   rdfs:domain ad:ResourceLevelAD ;
   rdfs:range ad:Intent .
ad:hasServiceIntent a owl:ObjectProperty ;
   rdfs:domain ad:ServiceLevelAD ;
   rdfs:range ad:Intent .
ad:hasSubIntent a owl:ObjectProperty ;
   rdfs:domain ad:Intent ;
   rdfs:range ad:Intent .
ad:hasUtility a owl:ObjectProperty ;
   rdfs:domain <http://example.org/AIAgent> ;
   rdfs:range ad:UtilityFunction .
ad:interADNegotiation a owl:ObjectProperty ;
```

© TM Forum 2025. All Rights Reserved.

```
  rdfs:domain ad:IntentManagementFunction ;
  rdfs:range ad:Negotiation .
ad:intraADNegotiation a owl:ObjectProperty ;
  rdfs:domain <http://example.org/AIAgent> ;
  rdfs:range ad:Negotiation .
ad:managesAIModel a owl:ObjectProperty ;
  rdfs:domain ad:API ;
  rdfs:range <http://example.org/AIAgent> .
ad:managesIntent a owl:ObjectProperty ;
  rdfs:domain ad:IntentManagementFunction ;
  rdfs:range ad:Intent .
ad:mapsToResource a owl:ObjectProperty ;
  rdfs:domain ad:ResourceLevelAD,
    ad:ServiceLevelAD ;
  rdfs:range ad:RFS .
ad:mapsToService a owl:ObjectProperty ;
  rdfs:domain ad:ServiceLevelAD ;
  rdfs:range ad:CFS .
ad:optimizesIntent a owl:ObjectProperty ;
  rdfs:domain <http://example.org/AIAgent> ;
  rdfs:range ad:Intent .
ad:performanceMetric a owl:DatatypeProperty ;
  rdfs:domain ad:UtilityFunction ;
  rdfs:range rdfs:Literal .
ad:usesAPI a owl:ObjectProperty ;
  rdfs:domain ad:AutonomousDomain ;
  rdfs:range ad:API .
ont:DiscoveryPhase a ont:IntentLifecyclePhase .
ont:FulfillmentPhase a ont:IntentLifecyclePhase .
ont:IntentFaultRecoveryAgent a owl:Class ;
  rdfs:subClassOf [ a owl:Restriction ;
      owl:onProperty ont:hasRuntimeMetric ;
      owl:someValuesFrom ont:RuntimeMetric ],
    [ a owl:Restriction ;
      owl:onProperty ont:hasIntentExpression ;
      owl:someValuesFrom ont:IntentExpression ],
    <AIAgent> .
ont:IntentHandlerAgent a owl:Class ;
  rdfs:subClassOf [ a owl:Restriction ;
      owl:onProperty ont:hasIntentExpression ;
      owl:someValuesFrom ont:IntentExpression ],
    [ a owl:Restriction ;
      owl:onProperty ont:hasRuntimeMetric ;
      owl:someValuesFrom ont:RuntimeMetric ],
    <AIAgent> .
ont:IntentMonitoringAgent a owl:Class ;
  rdfs:subClassOf [ a owl:Restriction ;
      owl:onProperty ont:hasRuntimeMetric ;
      owl:someValuesFrom ont:RuntimeMetric ],
    [ a owl:Restriction ;
      owl:onProperty ont:hasIntentExpression ;
      owl:someValuesFrom ont:IntentExpression ],
    <AIAgent> .
ont:IntentNegotiationAgent a owl:Class ;
  rdfs:subClassOf [ a owl:Restriction ;
      owl:onProperty ont:hasIntentExpression ;
      owl:someValuesFrom ont:IntentExpression ],
    [ a owl:Restriction ;
      owl:onProperty ont:hasRuntimeMetric ;
      owl:someValuesFrom ont:RuntimeMetric ],
    <AIAgent> .
ont:IntentOwnerAgent a owl:Class ;
  rdfs:subClassOf [ a owl:Restriction ;
```

© TM Forum 2025. All Rights Reserved.

```
        owl:onProperty ont:hasRuntimeMetric ;
        owl:someValuesFrom ont:RuntimeMetric ],
      [ a owl:Restriction ;
        owl:onProperty ont:hasIntentExpression ;
        owl:someValuesFrom ont:IntentExpression ],
      <AIAgent> .
ont:IntentTranslationAgent a owl:Class ;
  rdfs:subClassOf [ a owl:Restriction ;
        owl:onProperty ont:hasIntentExpression ;
        owl:someValuesFrom ont:IntentExpression ],
      [ a owl:Restriction ;
        owl:onProperty ont:hasRuntimeMetric ;
        owl:someValuesFrom ont:RuntimeMetric ],
      <AIAgent> .
ont:NegotiationPhase a ont:IntentLifecyclePhase .
ont:coordinatesWith a owl:ObjectProperty ;
  rdfs:domain <AIAgent> ;
  rdfs:range <AIAgent> .
ont:delegatesTo a owl:ObjectProperty ;
  rdfs:domain <AIAgent> ;
  rdfs:range <AIAgent> .
ont:initiatesPhase a owl:ObjectProperty ;
  rdfs:domain <AIAgent> ;
  rdfs:range ont:IntentLifecyclePhase .
ont:resolvesIntent a owl:ObjectProperty ;
  rdfs:domain <AIAgent> ;
  rdfs:range ont:IntentExpression .
ad:TMF620 a ad:API ;
  rdfs:label "Product Catalog Management API" .
ad:TMF621 a ad:API ;
  rdfs:label "Trouble Ticket Management API" .
ad:TMF645 a ad:API ;
  rdfs:label "Service Qualification Management API" .

ad:TMF679 a ad:API ;
  rdfs:label "Product Offering Qualification API" .
ont:AIAgentShape a sh:NodeShape ;
  sh:property [ sh:class ont:IntentExpression ;
        sh:minCount 1 ;
        sh:path ont:hasIntentExpression ],
      [ sh:class ont:RuntimeMetric ;
        sh:minCount 1 ;
        sh:path ont:hasRuntimeMetric ] ;
  sh:targetClass <AIAgent> .
ad:RFS a owl:Class .
ad:TMF633 a ad:API ;
  rdfs:label "Service Catalog Management API" .
ad:UtilityFunction a owl:Class .
ad:CFS a owl:Class .
ad:Negotiation a owl:Class .
ad:ResourceLevelAD a owl:Class ;
  ad:usesAPI ad:TMF621,
      ad:TMF633,
      ad:TMF641,
      ad:TMF915,
      ad:TMF921 ;
  rdfs:subClassOf ad:AutonomousDomain .
ad:TMF641 a ad:API ;
  rdfs:label "Service Ordering API" .
ad:TMF915 a ad:API ;
  rdfs:label "AI Management API" ;
  ad:managesAIModel <http://example.org/AIAgent> .
ad:TMF921 a ad:API ;
```

© TM Forum 2025. All Rights Reserved.

```
  rdfs:label "Intent Management API" .
ad:IntentManagementFunction a owl:Class .
ad:ProductLevelAD a owl:Class ;
  ad:usesAPI ad:TMF620,
    ad:TMF641,
    ad:TMF679,
    ad:TMF915,
    ad:TMF921 ;
  rdfs:subClassOf ad:AutonomousDomain .
ont:IntentLifecyclePhase a owl:Class .

ad:ClosedLoopControl a owl:Class .
ad:ServiceLevelAD a owl:Class ;
  ad:usesAPI ad:TMF633,
    ad:TMF641,
    ad:TMF645,
    ad:TMF915,
    ad:TMF921 ;
  rdfs:subClassOf ad:AutonomousDomain .
ad:AutonomousDomain a owl:Class .
ad:Intent a owl:Class .
ont:hasIntentExpression a owl:ObjectProperty ;
  rdfs:domain <AIAgent> ;
  rdfs:range ont:IntentExpression .
ont:hasRuntimeMetric a owl:ObjectProperty ;
  rdfs:domain <AIAgent> ;
  rdfs:range ont:RuntimeMetric .
ont:RuntimeMetric a owl:Class .
ont:IntentExpression a owl:Class .
ad:API a owl:Class .
<http://example.org/AIAgent> a owl:Class .
<AIAgent> a owl:Class ;
  rdfs:seeAlso ont:AIAgentShape ;
  owl:equivalentClass <http://example.org/AIAgent> .
```

© TM Forum 2025. All Rights Reserved.

9. Appendix B: MAS AI Agent Intent-Based AN Operations – Formulation and Analytics

The AI agent intent-based ANs enabled to dynamically interpret, enforce, and adapt to evolving services and resources real-time demands. Unlike traditional SLOs, where requirements remain static and do not propagate to existing instances, intent-driven operations use AI-agent intent-based IMF to ensure real-time compliance and adaptation.

AI agents, such as the intent owner agent, continuously refine and update intent requirements, ensuring that any changes immediately affect already deployed service and resource instances. This level of automation enhances network efficiency by allowing intent translator agents to convert high-level business goals into executable policies and intent handler agents to manage the full intent lifecycle, including validation, enforcement, and optimization.

SLOs are traditionally managed by human operators, whereas intent-based AI-driven systems allow AI agents to autonomously negotiate and enforce new requirements dynamically. These intent monitoring agents track network compliance and generate intent reports, ensuring alignment between network states and business goals. The shift to AI agent intent-based operations marks a critical advancement in network automation, transforming static management paradigms into intelligent, self-optimizing frameworks.

9.1. AI-Agent MAS AN

Autonomous networks are structured as ADs, as defined by the TMF IG1253. These ADs are distributed across various layers of the network operation stack—product, service, and resource layers. Within each layer, multiple AI agents work collaboratively, forming a MAS that ensures compliance with intent-driven requirements. In this hierarchical framework, ADs leverage intent management agents to coordinate inter and intra-domain interactions. Each AI Agent within a domain has a well-defined role, such as optimizing resources, enforcing security policies, or ensuring service quality. This architecture follows a structured approach:

- Product-Level AI Agents
  - Manage customer-facing services, ensuring business intent alignment.
- Service-Level AI Agents

© TM Forum 2025. All Rights Reserved.

    - Convert business requirements into actionable service-level configurations.
- Resource-Level AI Agents
    - Execute network orchestration by configuring and optimizing infrastructure resources.

Intent-based AI agent coordination enables an adaptive network where each domain sets and enforces requirements using standardized APIs. This process ensures seamless interaction between domains, fostering an intelligent and self-sustaining network ecosystem.

9.2. AI Agent Enabled Intent Management

Each AD integrates Intent Managers, IMF, realized through AI agent intent based cognitive loops, to coordinate intent-based operations. These IMF communicate using intent statements, ensuring network goals are met efficiently within AI agent different roles. The AI agent intent-based IMF leverage APIs to manage the entire lifecycle of intent.

TMF921 Intent Management API enables communication between AI Agents, facilitates intent registration, status tracking, and compliance monitoring. This API enables AI-driven automation by allowing systems to modify requirements dynamically, ensuring real-time adaptation to changing network conditions. AI Agents utilize TMF921 APIs to

- Efficiently manage intent-based automation without modifying operator-defined SLOs. TMF921 capabilities
- Modifies intent dynamically to optimize service delivery.
- Monitors compliance with both SLOs and intent-driven network policies.
- Handles feasibility checks to determine network capability in meeting intent-driven requirements.
- Facilitates inter-domain negotiations to optimize service and resource allocation.
- Continuously assess the network's compliance with intent-based objectives
- Ensures seamless interaction between Intent Owners and Intent Handlers
- Modifies intent to reflect evolving business needs.
- Reporting compliance status and identifies optimization opportunities.
- Handles feasibility checks to determine network capability in meeting intent-driven requirements.
- Facilitating inter-domain negotiations, seamless intent propagation.

9.3. AD Hierarchy Model

The hierarchy of ADs includes:

- Product-Level AD

© TM Forum 2025. All Rights Reserved.

  - Handles the commercial intent and breaks it down into CFS requirements.
- Service-Level AD
  - Translates CFS intent into Service Intent before forwarding it to Resource-Level ADs.
- Resource-Level AD
  - Implements RFS by orchestrating network, computing, and application resources.

Autonomous Domain Workflow

- Product-Level AD
  - Defines business intent for the 5G slice.
  - Maps to CFS (e.g., "Ultra-Reliable Low-Latency Service").
- Service-Level AD
  - Maps CFS into technical service requirements.
  - Assigns QoS policies (e.g., prioritization, traffic engineering).
- Resource-Level AD
  - Allocates network RFS for the service.
  - Implements edge computing **deployment** for latency reduction.

The intent processing model includes:

- Product-Level Intent Definition
  - I_P be the **product intent** at the Product Level AD.
  - CFS_k be the CFS fulfilling a given intent.
  - $U_P(I_P)$ be the **utility function** measuring the intent fulfillment.
- Each Product-Level AD defines a high-level intent:

$$I_P = \{(T_k, P_k)\}$$

where:

- T_k is the Target Delivery Expectation for CFS_k.
- P_k is the Property Expectation that must be met.
- The CFS mapping function transforms product intent into CFS:

$$CFS_k = f_P(I_P)$$

$$U_P = w_1 F_1(CFS) + w_2 F_2(CFS) + \cdots . + w_n F_n(CFS)$$

where w_i are weights for different product objectives (e.g., revenue, KPI compliance).

Service-Level Intent Definition: At the Service-Level AD, business intent is refined into technical service intent:

$$I_S = g_P(CFS)$$

© TM Forum 2025. All Rights Reserved.

The function g_P represents the intent transformation process from the CFS level to the Service-Level Intent **(I_S)** decomposition. g_P is a function that translates business intent from a product perspective into technical service **intent** decomposition. It maps CFS-level requirements (which are business-oriented) into service-level specifications (which are more technical and operational). The subscript P indicates that this function operates from the product level. The function g_P translates this into a Service-Level Intent I_S like: Low Latency ≤ 5ms, guaranteed Bandwidth ≥ 100 Mbps, and QoS Priority Level = High.

Suppose a CFS is "Premium 5G Connectivity", which is a commercial offering.

$$I_S = \{(T_m, P_m)\}$$

where:

- T_m = Target service-level expectation.
- P_m = Service quality requirement (e.g., latency, availability).

9.3.1. Intent Properties

The 'hasSubIntent' property defines a hierarchical relationship between intents, meaning that a higher-level intent can be decomposed into multiple sub-intents. The hasSubIntent provide the following:

- Intent Decomposition:
  - In an intent-driven model, a high-level business intent (e.g., "Provide 5G connectivity with ultra-low latency") must be broken down into smaller, more manageable intents across different domains.
  - Example:
    - "Optimize Network Slicing."
    - "Allocate Dedicated Bandwidth."
    - "Ensure Edge Computing Resource Availability."
    - High-Level Intent: "Ensure 5G Ultra-Reliable Low Latency Communication (URLLC)."
    - Sub-Intents
- Intent Processing at Multiple Levels:
  - A Product-Level Intent may generate multiple Service-Level Intents.
  - A Service-Level Intent may generate multiple Resource-Level Intents.
  - Each sub-intent is processed within its respective AD.

© TM Forum 2025. All Rights Reserved.

The hasSubIntent work in the model as follows:

1. Product-Level Intent (Handled in Product-Level AD)
    - hasSubIntent breaks it into multiple CFSs.

Service-Level Intent (Handled in Service-Level AD)

- hasSubIntent breaks it into multiple RFS.

3. Resource-Level Intent (Handled in Resource-Level AD)

- hasSubIntent ensures the intent is mapped to resource allocations.

9.4. Service Utility Function

The Service Utility Function evaluates how well the service-level intent is met:

$$U_S = w_1 F_1(I_s) + w_2 F_2(I_s) + \cdots .. + w_n F_n(I_s)$$

where $F_i(I_s)$ measures KPI adherence, service orchestration efficiency, and cost.

- Resource-Level Intent Definition

At the Resource-Level AD, service intent decomposition is mapped to RFSs:

$$RFS_q = h_S(I_s)$$

This function h_S represents the mapping of service-level intent decomposition into resource-level intent. h_S translates service intent into RFSs, which are directly bound to the underlying infrastructure (network functional elements). The subscript S indicates that this function operates from the service layer to the resource layer. Suppose a Service-Level Intent is "Low-Latency 5G Data Routing". The **function** h_S translates this into resource-specific RFS, such a Deploy Edge computing node (to reduce latency), assign 100 MHz Spectrum in the 3.5 GHz Band (to meet bandwidth needs).

$$I_R = \{(T_q, P_q)\}$$

where:

- T_q = Resource target expectation (e.g., allocated bandwidth, compute capacity).
- P_q = Technical constraints (e.g., network routing policies).

© TM Forum 2025. All Rights Reserved.

The Resource Utility Function ensures resource efficiency and network optimization:

$$U_R = w_1 F_1(RFS) + w_2 F_2(RFS) + \cdots . + w_n F_n(RFS)$$

where $F_i(RFS)$ considers network load, energy efficiency, and service orchestration feasibility as shown in Table **27**.

Table 27: CFS and RFS Function Transformation

Function	Transformation	Meaning
g_P	$I_S = g_P(CFS)$	Transforms CFS (business-level service) into service-level intent
h_S	$RFS_q = h_S(I_S)$	Transforms service-level intent into RFS

9.4.1. Autonomous Domain Closed-Loop Control Mechanism

The closed loop control is represented as a sequence of functional mappings intra and inter layers:

- $BusinessToServiceFlow: BusinessIntent \rightarrow ServiceIntent$
- $ServiceToResouceFlow: ServiceIntent \rightarrow ResouceIntent$
- $ResouceToBusinessFlow: ResourceIntent \rightarrow BusinessIntentReport$

Closed loop control consist of intent to communicate and distribute requirements and intent reports to close the loop with feedback and allows negotiation of requirements, thus collaborative refinement and optimization of intent.

The intra closed loops are the AD closed loops, and these domains can be either Business, Service and Resource layer, these are the loops implemented and coordinated by IMFs. Each AD implements a Closed-Loop Control System with four key phases:

- Awareness Phase
  - Each AD continuously monitors its environment to detect changes in network conditions, SLA performance, and customer demand:

    $$S_t = \{CFS_t, RFS_t, Network_t\ , Computing_t\}$$

    where S_t represents the real-time state of the system at time t.

- Analysis Phase
  - AI agents analyze the gap between actual performance and intent expectations:

    $$\Delta I = I_{Expected} - I_{Observed}$$

  - where:

© TM Forum 2025. All Rights Reserved.

- $I_{Expected}$ = Original product intent targets.
- $I_{Observed}$ = Measured system performance.

- Decision-Making Phase
  - AD selects the optimal corrective action:

$$Act_t = \underset{Act}{argmax}\, U(Act|S_t)$$

where:

- Act_t is the selected corrective action.
- $U(Act|S_t)$ measures how well the action improves intent fulfillment.

- **Execution Phase**
  - The selected action is applied to adjust the network or service configuration.

9.5. Cross-Domain Coordination and Negotiation Model

Agents interact within a domain to coordinate local domain intent, collaborate within the same domain to perform distributed tasks the so-called horizontal Interactions, or intra domain interaction. Agent also interacts across domains to exchange information to fulfill the global intent, exchange information across domains to support end-to-end service delivery, the so-called v**ertical domain interactions, or inter domain interaction**.

Horizontal Interactions
- Within a single AD, multiple AI agents collaborate to manage CFS or RFS execution.
- For example, multiple Resource-Level ADs coordinate spectrum allocation and network routing to optimize service delivery.

Vertical Interactions
- Product-Level AD → Service-Level AD → Resource-Level AD
- Each AD refines the intent before passing it down to the next level.

9.5.1. Intent Negotiation Game

Each AD participates in a cooperative game where it negotiates with other ADs to optimize shared objectives:

$$max \sum_{l} U_l$$

subject to:

- Cross-domain SLA constraints.
- Resource availability limits

© TM Forum 2025. All Rights Reserved.

5G Network Slice Orchestration example: a 5G provider provisions a network slice for an enterprise customer with:

- Guaranteed 5ms latency.
- Minimum 100 Mbps bandwidth.
- Closed-Loop Monitoring
  - AI agents continuously monitor latency.
  - If latency exceeds 5ms, AI triggers resource reallocation.

9.6. Intent Management Function (IMF) Model

The **IMF** model components include:

- Coordinates AI agents within a single AD (intra-AD coordination).
- Manages interactions between different ADs (inter-AD coordination).
- Handles intent propagation, negotiation, and optimization across hierarchical layers.

Entities & Definitions

Let

- AD_i - Autonomous Domain (e.g., Access, Transport, Core, etc.,).
- IMF_i - IMF within domain AD_i.
- $Ag_{i,j}$ - is the jth agent that support a specific function in AD_i.
- I_i - is the **intent** that is managed by IMF_i within AD_i.
- CFS - CFS is managed by higher-level ADs.
- RFS - RFS is managed at the resource level.

Each **IMF**:

- Coordinates multiple AI agents within its AD (intra-AD coordination).
- Negotiates with IMFs of other ADs (inter-AD coordination).
- Ensures closed-loop decision-making for self-optimization and autonomous control.

9.6.1. Formalizing Intent Processing in IMF

Each *IMF* interprets, refines, and executes intent:

$$I_i = \{(T_k, P_k)\}$$

where:

- T_k = Target expectation (e.g., bandwidth allocation, latency).
- P_k = Property constraint (e.g., QoS requirement, SLA).

The IMF decomposes intent into service or resource-level requirements respectively:

© TM Forum 2025. All Rights Reserved.

$$S_i = f(I_i)$$

where S_i represents CFS or RFS.

Each IMF applies an optimization function to determine the best actions:

$$Act_i = \underset{Act}{argmax}\, U(Act|S_i)$$

The above formula is a mathematical representation of decision-making optimization used in autonomous network operations with AI-driven agents. It describes how an AI Agent selects the optimal action Act_i to maximize the utility function $U(A|S_i)$ given the current system state S_i.

where:

- Act_i → The action chosen by the AI agent at time i by the IMF
- $\underset{Act}{argmax}$ → find the argument (action) Act that maximizes the function.
- Act → The set of all possible actions the AI agent can take.
- $U(Act|S_i)$→ The utility function, which measures the expected benefit **or** performance of acting, Act when the system is in state S_i.
- S_i → The current state of the network, system, or resource domain

Interpretation in AI and Autonomous Networks. This equation is a RL-based decision rule where an AI agent:

- Observes the network state S_i which could include bandwidth usage, latency, failures, or service quality.
- Evaluates possible actions Act —such as rerouting traffic, allocating additional bandwidth, adjusting QoS settings, etc.
- Selects the action that maximizes the expected utility $U(Act|S_i)$ ensuring that intent-based service requirements are met while optimizing network performance.

Example in Intent-Driven Networks Optimization

AI Agent manages bandwidth allocation. Given a state S_i (e.g., high traffic congestion on a link), the agent considers multiple possible actions:

- Act_1 : Increase bandwidth on the congested link
- Act_2: Reroute traffic through an alternative path
- Act_3: Reduce non-priority traffic through policy adjustments

Each action has an associated utility value based on its expected impact on latency, packet loss, and overall service quality. The AI agent then chooses the action Act_i that maximizes utility, ensuring that the network operates efficiently while adhering to business and service-level intents.

© TM Forum 2025. All Rights Reserved.

9.6.2. MAS for Intra-AD Coordination

In a MAS, multiple AI agents can negotiate and collaborate using this decision-making principle. Each agent independently applies the formula but also interacts with other agents to align their intent-based objectives. For example, an Intent Negotiation Agent may select an action that not only maximizes its local utility but also contributes to a globally optimal network state, facilitating inter-domain coordination.

Each *IMF* manages multiple AI agents within a single AD_i:

$$Ag_{i,j} \in AD_i, \qquad j = 1,2, \dots \dots, n$$

where $Ag_{i,j}$ is an AI agent responsible for executing a specific function.

Each AI agent follows an MDP model:

$$M_{i,j} = (S, Act, T, R)$$

where:

- *S* = State of the network resources and service performance.
- *Act* = Action set for adjusting resources (e.g., bandwidth allocation, routing).
- *T(S, Act, S')* = State transition probability after action, *Act*.
- *R(S, Act)* = Reward function measuring intent fulfillment.

The optimal policy $\pi_{i,j}$ for each agent is:

$$\pi_{i,j}^* = arg \max_{\pi} \sum_{t=0}^{T} \gamma^t R(S_t, A_t)$$

where γ is the discount factor for balancing short-term vs. long-term gains.

Each IMF coordinates AI agents via Multi-Agent RL (MARL)**:**

$$\max_{\{A_{i,j}\}} \sum_{j=0}^{n} U_{i,j}$$

subject to:

- Resource constraints within AD_i.
- ddds.

9.6.3. Inter-AD Coordination (IMF-to-IMF Negotiation Model)

Each *IMF* interacts with other *IMF*s for cross-domain intent coordination.

© TM Forum 2025. All Rights Reserved.

$$IMF_i \leftrightarrow IMF_j \quad i \neq j$$

where IMF_i and IMF_j belong to different Autonomous Domains.

Intent Propagation Across ADs

Each *IMF* negotiates with others using a game-theoretic approach:

$$max \sum_{i=1}^{N} U_i$$

subject to:

- Cross-domain constraints (e.g., transport domain availability).
- Network-wide SLA compliance.

IMFs follow a multi-agent negotiation strategy:

- Propose intent adjustments if resource allocation is suboptimal.
- Negotiate service trade-offs (e.g., prioritizing ultra-low latency over bandwidth).
- Reach equilibrium where no IMF can improve without impacting another.

Example: Access, Transport, and Core Network Resource Coordination

The customer request is received at the product layer *IMF*, which is decomposed to *CFS*s and *RFS* (aka Network Service), then it is passed to the Service layer *IMF*. These are different ADs and the relevant *IMF* in each domain at the boundary of these domains perform the integration between these two ADs. Intent based *IMF* at the service layer enable the end-to-end integration programmatically through open APIs and data modelling tools for the access AD, Transport AD and Core AD as follows:

- Access AD, IMF_A detects congestion → requests additional bandwidth.
- Transport AD IMF_T negotiates bandwidth allocation from Core AD IMF_C.
- Core AD IMF_C adjusts network slicing to accommodate demand.
- IMFs finalize agreement and execute new network configuration**.**
- *managesIntent*: Links IntentManagementFunction to **Intent,** ensuring that each IMF is responsible for interpreting and handling intent**.**

- engagesInNegotiation: Links IntentManagementFunction to Negotiation, reinforcing IMF-based intent coordination across domains.
- IMF is also linked to Autonomous Domains **(***hasIMF***).**
- Agents **(**AIAgent**)** remain responsible for optimizing Intent **(***optimizesIntent*)
- Intent properly flows across Product, Service, and Resource levels **(**hasProductIntent**,** hasServiceIntent**,** hasResourceIntent**).**

© TM Forum 2025. All Rights Reserved.

IMFs should ensure:

- Autonomous resolution of resource conflicts.
- Optimal allocation of bandwidth across ADs.
- Dynamic adaptation based on real-time conditions.

9.6.4. Utility-Based Closed-Loop Control

Each *IMF* continuously monitors, analyzes, decides, and executes (MADE loop)**:**

Awareness (State Monitoring)

$$S_t = \{CFS_t, RFS_t, Resource_t, Network_t\}$$

- Real-time observation of network performance and service demand**.**

Analysis (Deviation Calculation)

$$\Delta I = I_{Expected} - I_{Observed}$$

- Measures the gap between expected and actual intent fulfillment**.**

Decision-Making (Action Optimization)

$$A_t = arg\max_{Act} U(Act|S_t)$$

- *IMF* selects the best action based on network constraints and SLA requirements.

Execution (Autonomous Policy Update)

- Applies AI-driven changes to the network, services, or resources.

Each *IMF* follows a **reward function**:

$$U = w_1F_1 + w_2F_2 + \cdots . + w_nF_n$$

where:

- F_1 = SLA compliance.
- F_2 = Latency reduction.
- F_3 = Cost efficiency.
- F_n = Other performance metrics.

The goal is to maximize global network efficiency:

$$\sum_{i=1}^{N} U_i \rightarrow max$$

© TM Forum 2025. All Rights Reserved.

9.6.5. 5G AI Agent Intent-Based Reports

Let's consider an **AI-driven network slicing use case** in a 5G autonomous network:

Scenario: A Service-Level Intent specifies:

- Guaranteed 100 Mbps bandwidth for a specific service.
- Maximum latency of 10 ms.
- Redundancy requirements for fault tolerance.

AI Agent Intent-Based Report Generation

- Nodes - AI Agents, network devices, service policies.
- Edges - Relationships between service intents, SLA compliance, and infrastructure.
- Attributes - Metrics like current bandwidth, latency, and fault status.

A partial graph snapshot (intent report) at time *T* might show:

- The network slice currently operates at 90 Mbps (below the required threshold).
- Latency has increased to 12 ms, indicating potential degradation.
- The Intent Monitoring Agent detects the issue and triggers an update.

The Intent Negotiation Agent might then reconfigure resources or request changes from the Resource-Level AI Agents.

We use Knowledge Graphs (KG) for AI Intent Reporting to enable

- Structured Representation - Provides a semantic model that integrates AI decision-making.
- Real-Time Monitoring - Captures the current state dynamically.
- Automated Inference - Enables AI reasoning to predict and proactively adjust network configurations.
- Inter-Agent Collaboration - Facilitates seamless coordination between AI agents in MAS.

AI Agent Report Function

- AI Agent MAS Intent-Based Report
    - Intent Reports in an AI-driven Intent Management System are not just static logs but evolving knowledge objects—integrated within a KG that continuously refines network operations through AI reasoning, multi-agent coordination, and real-time adaptation. An AI agent intent-based report as a KG represents the current state of an intent-driven AN in a structured format. The partial graph focuses on only the relevant aspects of the system at a given point in time, ensuring real-time compliance tracking, policy adaptation, and network self-optimization. This enables AI-driven intent-based service management to continuously monitor, adapt, and optimize network operations autonomously.
    - An AI agent intent-based report is structured as a (or a partial graph) because it represents the intent management state in a way that captures relationships, dependencies, and evolving network

© TM Forum 2025. All Rights Reserved.

conditions at a specific point in time. The following subsection provides explanation of how this concept works.

Generate Report Function:

The function generates an intent report

The function $GenerateReport(i, r)$ generates an intent report

$r \in IntentReport$ for intent $i \in Intent$

$$GenerateReport(i, r) = \{ReportData \mid SatisfiesRequirement(i)\}$$

i - refers to a specific intent

r - refers to a report that is generated for a specific intent *i*

© TM Forum 2025. All Rights Reserved.

10. Appendix C: Simulation vs. Emulation vs. Digital Twin – Tradeoffs in Resource-Level Coordination

Simulation and emulation are often referenced interchangeably in AI-assisted network automation, but their technical scope, realism, and applicability differ significantly. This appendix compares these methods along with the concept of digital twins, clarifying their roles and tradeoffs in the resource layer of intent-based autonomous networks (ANs) [21],[34].

Conceptual Relationships – Figure 11 is a Venn diagram that illustrates the conceptual and functional overlap between simulation, emulation, and digital twins in resource-level orchestration:

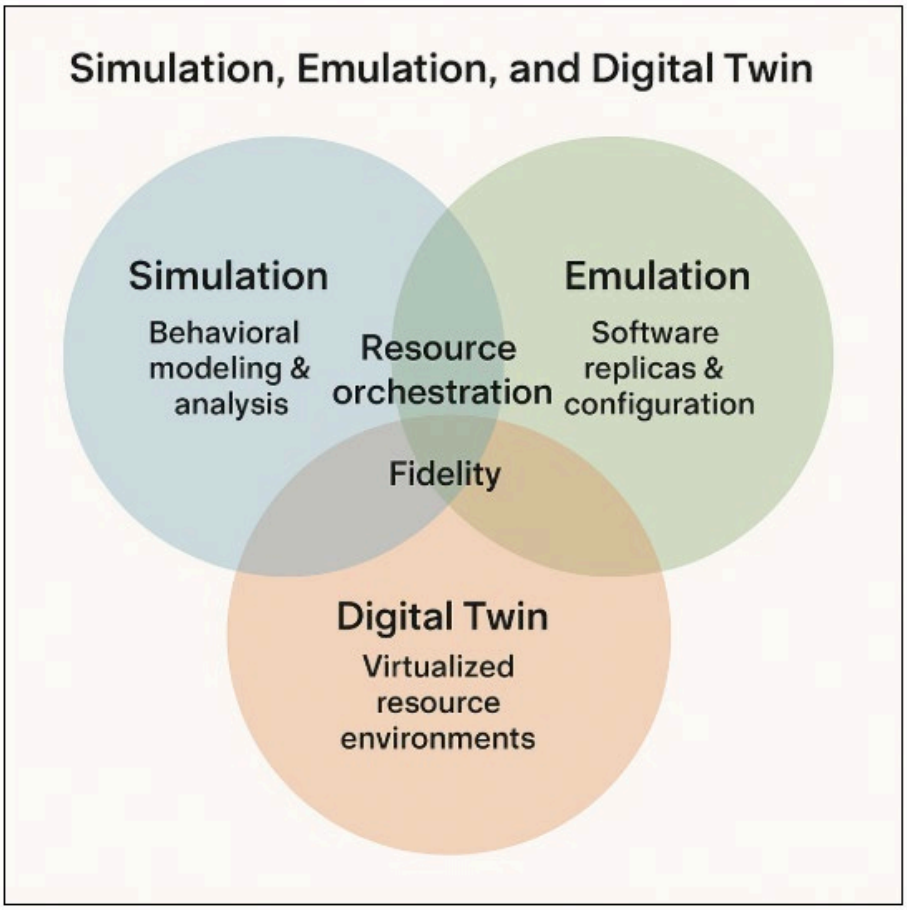

Figure 11: Three concepts intersect and complement one another in resource-level orchestration.

© TM Forum 2025. All Rights Reserved.

Simulation is useful for testing intent and policy constraints using abstracted models.

Emulation replicates real network behavior, providing protocol accuracy but lacks real-time telemetry.

Digital Twins offer a composite view by leveraging network topology, configuration, and service mappings to inform both simulation and emulation.

Key Interpretations:

- Simulation ∩ Digital Twin: Enables policy verification over modeled topologies (e.g., TMF686).
- Emulation ∩ Digital Twin: Enables protocol testing within a mirrored network model.
- Simulation ∩ Emulation: Combines model planning with real-behavior execution.
- All Three: Represents the ideal state of closed-loop assurance and dynamic intent evaluation.

10.1. Hybrid Telemetry / Partial-Twin Flow

Figure 12 depicts hybrid telemetry and partial twin flow as outlined below and summarize in Table 28:

1. Southbound telemetry (teal) flows upward from each network element to an Operational Data Broker.
2. The broker normalizes vendor YANG into the Dynamic Service & Inventory Map (orange).
3. The Ontology / SKOS Layer (blue) abstracts cross-vendor concepts.
4. Policy & Simulation Engines (violet) analyze the semantic graph, simulate what-if scenarios, and send northbound feedback (grey dotted arrows) as policy or intent adjustments back toward the resource-view.

© TM Forum 2025. All Rights Reserved.

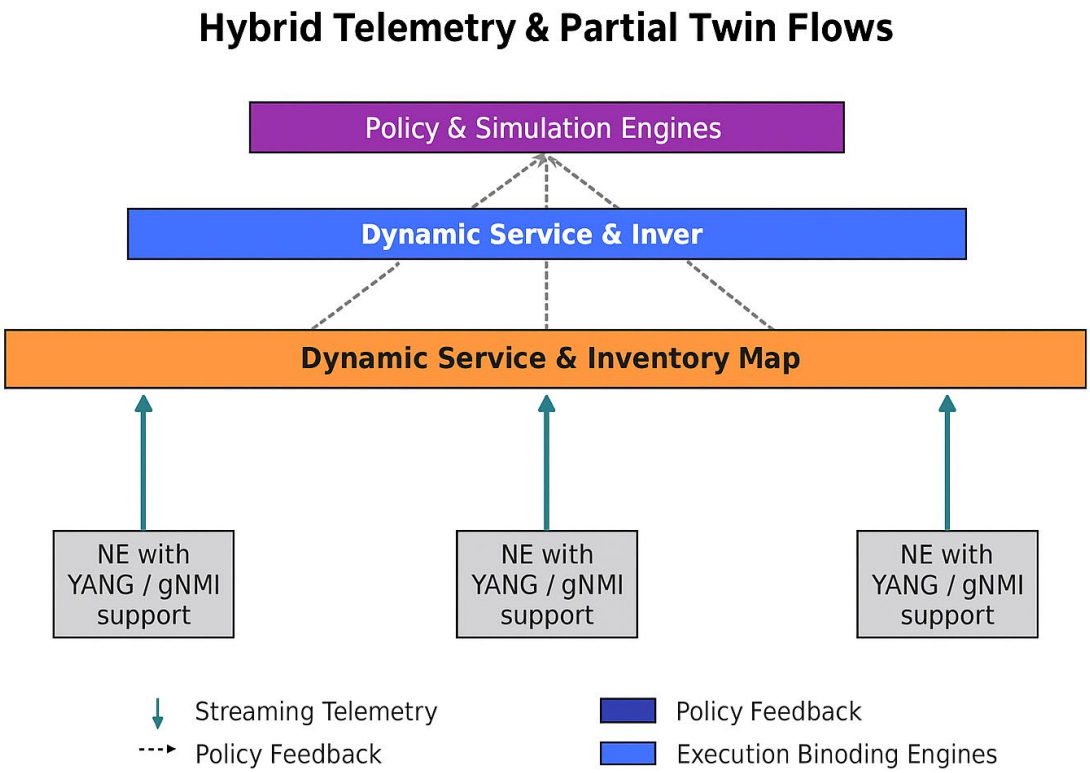

Figure 12: Hybrid Telemetry and Partial Twin Flow

Materialization Performance

While the partial twin model supports ingesting normalized telemetry (e.g., from YANG Push or gNMI), the architecture does not require full materialization at ingestion. Instead, materialization of knowledge graph triples can be deferred or selectively applied:

- Telemetry-first ingestion into time-series or NoSQL stores
- On-demand graph projection using virtual views or cached partial graphs
- SHACL-based validation for intent-relevant shapes, avoiding full ontology reasoning at load time
  This hybrid strategy balances expressiveness and performance, especially when integrating AI agents with high-frequency data sources across resource domains.

Table 28: Diagram Layer Outline Description

Diagram Layer / Arrow	Description in Diagram	Correspondence in Write-up
Teal Arrows (↑)	Streaming telemetry from devices	"Southbound telemetry (teal) flows upward..."

© TM Forum 2025. All Rights Reserved.

Diagram Layer / Arrow	Description in Diagram	Correspondence in Write-up
Orange Layer	Dynamic Service & Inventory Map	"Broker normalizes vendor YANG into..."
Blue Layer	Ontology / SKOS abstraction layer	"Ontology abstracts cross-vendor concepts..."
Violet Box	Policy & Simulation Engines	"Engines simulate what-if scenarios..."
Grey Dotted Arrows (↓)	Policy feedback / adjustments	"Send northbound feedback... as policy or intent adjustments..."

10.1.1. Ontology Alignment

Your ontology constructs the Intent Translator Module as outlined below and summarized in Table 30:

1. **Intent Expression Ontology Class**

- Captures business/technical goal, constraint, and KPI semantics
- Formalizes IntentStatement, IntentGoal, IntentConstraint, and IntentTarget

2. **Intent Context & Domain Binding**

- Uses hasScope, appliesTo, and hasDomain properties to define:
  - Where the intent applies (Product, Service, or Resource AD)
  - The scope for resolution (e.g., slice, function, NF set)

3. **Intent Transformation Rules**

- Defined using IntentToPolicyMapping or IntentActionPattern
- Includes transformation logic modeled as part of **semantic rule base** (e.g., via SHACL, SWRL, RIF)

Table 29: AI Agent Module Architecture

Subcomponent	Function
Intent Parser	Extracts declarative fields from natural/structured input (goal, metric)
Ontology Binder	Resolves parsed tokens to ontology classes (IntentGoal, IntentDomain, etc.)
Semantic Mapper	Translates intent graph patterns into candidate policies or orchestration patterns
Policy Generator	Forms actionable rule statements using predefined templates (e.g., PBC, TMF rules, YANG mapping)
Feedback Connector	Integrates with AI Monitoring Agent to learn translation efficacy and adapt mappings

© TM Forum 2025. All Rights Reserved.

11. Appendix D: Integration of AI Agent MAS Framework

The **Intent Translator Agent**:

- Operates at both **Product and Service Domains**
- Interacts upstream with the **Intent Owner Agent**
- Outputs machine-readable policies to **Intent Handler Agent**
- Using the **ontology** as the semantic grounding model
- May use pre-trained **LLMs** or symbolic reasoning engines (e.g., SPARQL, SWRL) to resolve ambiguity or refine intent as shown in Table 30

Table 30: Formal Representation Snippet (OWL/Turtle-Aligned)

```
:IntentTranslatorAgent a :AIAgent ;
  :hasFunction :TranslateIntent ;
  :usesOntology :IG1421IntentModel ;
  :bindsIntent [
    a :IntentStatement ;
    :hasGoal :EnsureAvailability ;
    :hasConstraint :AvailabilityGT99_99 ;
    :appliesTo :ServiceSliceX ;
  ] ;
  :outputs [
    a :Policy ;
    :appliesTo :ServiceSliceX ;
    :definesTargetState :HighAvailabilityConfigured ;
  ] .
```

11.1.1. Alignment with MI CONOPS 2035-2050 Insight [40]

The MI CONOPS 2035–2050 roadmap [40] emphasizes intelligence modernization through AI augmentation, multi-agent autonomy, and cognitive orchestration, aligning strongly with the intent-based MAS architecture presented in this document.

Intent as Declarative Control: The concept of expressing mission goals through declarative intents mirrors the telecom shift to declarative service management. Agent-Based Reasoning: AI agents contexts should perform proactive reasoning, prediction, and adaptation using knowledge bases—driven by structured observations and mission/intent specifications.

Dynamic Autonomy and Human-AI Teaming: The telecom MAS facilitates real-time decision support and domain coordination without centralized orchestration. Cross-Domain Collaboration: Interoperability across autonomous domains (in telecom and operations service/resource stacks) is a shared architectural requirement.

© TM Forum 2025. All Rights Reserved.

Intent Lifecycle Automation TMF IG1253C [41]: The roadmaps propose end-to-end lifecycle control of missions; this aligns with IMF intent cycle management within AI-driven telecom infrastructure.
These parallels validate the design maturity of the AI agent MAS model and underscore its applicability across sectors seeking resilient, real-time, and intent-driven automation.

11.1.2. Materialization Performance

While the partial twin model supports ingesting normalized telemetry (e.g., from YANG Push or gNMI), the architecture does not require full materialization at ingestion. Instead, materialization of knowledge graph triples can be deferred or selectively applied:

- Telemetry-first ingestion into time-series or NoSQL stores
- On-demand graph projection using virtual views or cached partial graphs
- SHACL-based validation for intent-relevant shapes, avoiding full ontology reasoning at load time
  This hybrid strategy balances expressiveness and performance, especially when integrating AI agents with high-frequency data sources across resource domains.

© TM Forum 2025. All Rights Reserved.

12. Appendix E: Carrier-Influenced BGP Path Selection

Modern wide-area networks rely on BGP not only within an operator's infrastructure but between carrier networks. The BGP best-path algorithm (RFC 4271) evaluates a sequence of attributes—many of which (e.g., LOCAL_PREF, route communities, AS-path prepends) embody *commercial or policy constraints* imposed by upstream and downstream providers. Consequently, the chosen path is frequently the most policy-compliant path, not the shortest. Any meaningful simulation or partial digital-twin that aims to predict traffic shifts must therefore:

- Reproduce the standards-defined decision process **and**
- Ingest at least a proxy of external-carrier attributes or historical observations, because these external policies materially affect the routes visible inside one's own administrative domain.

This reinforces why today's simulations typically adopt a *hybrid* approach—combining standards logic, vendor YANG models, and observed BGP attributes—to approximate behavior, while recognizing that 100 % fidelity remains an open challenge.

© TM Forum 2025. All Rights Reserved.

13. Appendix F: Best Practices

Table 31 summarizes Simulation, emulation, and digital twin comparison

- Use simulation for intent translation, resource planning, and validation against topology constraints.
- Use emulation to test device configurations or routing behavior in isolated or pre-production settings.
- Use domain-scoped digital twins for visual overlays, service-mapping verification, and proactive assurance.

Table 31: Simulation, Emulation and Digital Twin Comparison

Capability	Simulation	Emulation	Digital Twin (Current State)
Definition	Modeling hypothetical scenarios using abstracted representations	Replicating real network devices or software for realistic behavior	Logical overlay mirroring partial network state, topology, and behavior
Strengths	Lightweight, scalable; useful for intent/policy validation and what-if analysis	Accurate representation of protocol/device behavior	Topology visualization; service-impact validation; closed-loop assurance foundation
Limitations	Lacks vendor-specific protocol behavior; can't simulate full control plane logic	High setup cost; limited real-device software availability for access networks	Not real-time; may lack behavioral fidelity; depends on accurate inventory/topology data
Use Cases	Intent validation, path planning, policy conflict testing	Route convergence testing, configuration migration, protocol compliance checks	Topology overlay validation, service-to-resource impact analysis, intent realization checks
Integration Potential	Easily integrates with orchestration and policy APIs (e.g., TMF686)	Best in labs or testbeds; not ideal for dynamic live-data use	Composable with TMF APIs; supports ZTP, assurance overlays, and service simulation

© TM Forum 2025. All Rights Reserved.

14. Appendix G: T-Box and A-Box

In ontology and knowledge representation (especially in Description Logics, OWL, and semantic web contexts), **T-Box** and **A-Box** (see Table 32) are two foundational components used to structure knowledge:

14.1. T-Box (Terminological Box)

- Defines the vocabulary and schema of the domain.
- Describes concepts (classes) and relationships (properties) between those concepts.
- Think of it as the **ontology's "dictionary"** or **schema**.
- Example:
  - Router ⊑ NetworkDevice (Every router is a network device)
  - hasPort: NetworkDevice → Port

14.2. A-Box (Assertional Box)

- Contains assertions about individual instances of the concepts defined in the T-Box.
- Captures factual data (i.e., the current state of the world).
- Example:
  - Router1: Router (Router1 is an instance of the class Router)
  - Router1 hasPort Port5

Table 32: Summary of T-Box and A-Box

Aspect	T-Box	A-Box
Role	Schema / vocabulary	Instance data / facts
Describes	Classes, relationships	Individuals, class membership
Example	Switch ⊑ NetworkDevice	Switch42: Switch
Analogy	Table definition in a database	Rows/records in the database table

Cautionary Note on Ontology (TBox) and Instance Data (ABox) in Knowledge Graphs

While the terminology of TBox (schema-level knowledge) and ABox (instance-level data) is standard in ontology literature, it is often unfamiliar outside the semantic web community. Therefore, it's clearer to refer to them collectively as components of the knowledge graph, distinguishing between schema and instance data when necessary.

© TM Forum 2025. All Rights Reserved.

Importantly, dynamic updates to the schema (TBox)—such as introducing new classes, properties, or changing existing relationships—can lead to reclassification or invalidation of existing instance data (ABox). In such cases, a partial or full rewrite of affected triples may be required. This poses significant performance implications, especially for large graphs.

Graph database performance also depends on the inferencing strategy:

- Materialization at ingestion (used in many open-source triple stores) can be computationally expensive during updates, as all inferences must be recomputed.
- Query-time inferencing, on the other hand, defers reasoning to runtime, and may be better suited for environments where instance data updates (e.g., telemetry from networks) occur frequently.

In practice, learning how inferencing works during ingestion can deepen understanding of the ontology's expressive power, but for high-frequency data updates, a query-time inference model is more scalable and responsive.

© TM Forum 2025. All Rights Reserved.

15. Appendix H: Standards-Aligned Intent Execution Template

This template is designed to ensure semantic clarity, operational traceability and interoperability between system expressing and realizing intents.

15.1. Intent Model RFC 9315 and RFC9316

1. Intent Declaration Block

(From RFC 9315 & RFC 9316: semantic classification + declarative form). This block is Fully supported by TMF TR290A [37]. The Intent model is outlined in Table 33

Table 33: Intent Model

Field	Description
intent-id	Globally or domain-unique identifier for the intent
intent-class	High-level classification: e.g., Optimization, Protection, Assurance, Experience, ResourceUtilization
actor	Who originated the intent (e.g., Operator, Application, Consumer, Orchestrator)
objective	Declarative goal (e.g., “latency < 20ms”, “maximize throughput”, “reduce signaling congestion”)
priority	Priority level (e.g., high, medium, best-effort
temporal-context	Optional: duration or validity window (start-time, end-time, event-triggered)

- TMF TR290A [37] directly supports this block.
- It defines fields such as:
  - actor
  - intentType
  - objective
  - priority
  - scope
  - target
- TR290 [36] provides the conceptual definitions and relationships.

Fully supported by TMF TR290A [37]

© TM Forum 2025. All Rights Reserved.

2. Policy & Semantic Parsing Layer

Table 34 shows the intent field description *(RFC 9315 Section 6: Policy derivation and validation logic)*

Table 34: Intent Field Description

Field	Description
Target-objects	Network elements, services, or logical entities to which the intent applies
resolved policy	Derived policy rules (e.g., threshold rules, placement constraints, routing hints)
validation-shape	SHACL/JSON schema ensuring conformance (optional)
intent-type	RFC 9316 Taxonomy type: e.g., Declarative, Query, Trigger, Control, Goal
intent-subtype	More specific form: e.g., ThresholdIntent, PerformanceIntent, PlacementIntent

- TR290A [36] defines semantics and context but does **not** provide a formal mapping or inference logic.
- TMF lacks formal definitions for:
  - Policy resolution engines
  - Semantic parsing rules
  - Rule-based vs ML-based translation

Partially supported in TMF. RFC 9315 and ETSI TS 104 036 support parsing and policy realization models.

3. Execution Binding Block

The intent functional blocks are summarized in Table **35** *(ETSI TS 104 036: Execution within RAN or infra domain)*

Table 35: Intent Functional Block

Field	Description
execution-agent	The functional block responsible for acting on the intent (e.g., AI agent, SMO function, RIC xApp, SDN controller)
mechanism	Execution type: AI-inferred, Rule-based, Closed-loop, Predefined-scenario
validation-shape	SHACL/JSON schema ensuring conformance (optional)
actuation-layer	KPIs or telemetry used to verify success (e.g., SLA latency, throughput, energy profile)
fallback-policy	Optional: action if the intent cannot be fulfilled (e.g., retry, notify, degrade gracefully)

© TM Forum 2025. All Rights Reserved.

- TR290 [36] and TR290A [37] stop at intent declaration.
- TMF does not yet define how execution agents (e.g., AI agents, orchestrators, SMO functions) bind to intent).
- TMF Open APIs (like TMF641, TMF642) can support actuation, but they're not part of TR290 [36].

Not defined in TR290 [36]. Use IETF RFCs and ETSI 104 036 as authoritative references here.

4. Runtime Monitoring and Assurance Block

Table 36 shows the observability points *(Supports feedback and closed-loop adaptation)*

Table 36: Observability Points

Field	Description
observability-points	Points in the system where intent compliance is monitored
violation-handlers	AI/analytics triggers for remediation
feedback-loop-id	Loop controller tracking this intent's realization lifecycle
status	Current lifecycle state: active, violated, suspended, fulfilled, expired
update-policy	How frequently the intent realization state is re-evaluated (e.g., interval, event-driven)

- TR290B [37] support intent reporting.
- It supports fields such as:
  - status
  - metrics
  - compliance result
  - violation state
  - feedback loop ID

Supported by TMF TR290B.

5. Example: Performance Optimization Intent for RAN (Table 37)

Table 37: Performance Optimization Intent for RAN

Performance Optimization Intent for RAN
{ "intent-id": "intent-ran-optimize-latency", "intent-class: "Optimization", "scop": RANFunction", "actor": "OperatorPolicyEngine", "objective": "latency < 10ms", "priority: "high", "intent-type": "Goal",

© TM Forum 2025. All Rights Reserved.

```
“intent-subtype”; ThresholdInent”,
“execution-agent”: “RIC-xApp”,
“mechanism”; AI-inferred”,
“actuation-layer: “RAN”,
“compliance-metric”: [“latency-RTT”, UE-packet-delay”]
“status”; “active”,
“observability-points: [“gNodeB”, “RIC-KPI-Agent”],
“fallback-policy”: “alert + degrade-QoS-profile”,
“feedback-loop-id”: loop-ran-001”
}
```

15.2. TMF Intent Model Artifacts and Alignment

TMF intent artifact are outlined in Table 38.

Table 38: TMF Intent Artifact

TMF Doc ID	Title	Version	Purpose / Role in Intent Lifecycle	Alignment
TR290 [36]	Intent Common Model	v3.0.0	Provides an overview and common definitions of intent structure and lifecycle.	Aligns with RFC 9315's semantic model foundation; defines shared ontology elements.
TR290A [37]	Intent Common Model – Intent Expression	v3.0.0	Defines the model elements used for declaring intent (e.g., goal, actor, target, priority).	Direct mapping to intentType, objective, and scope fields from RFC 9316.
TR290B [38]	Intent Common Model – Intent Reporting	v3.0.0	Describes how to express intent realization and compliance through reporting constructs.	Aligns with SHACL-based validation, A-Box compliance feedback, and observability loops.

© TM Forum 2025. All Rights Reserved.

16. Terms & Abbreviations Used within this Document

16.1. Terminology

Term	Definition	Source

16.2. Abbreviations & Acronyms

Abbreviation / Acronym	Abbreviation/Acronym Spelled Out	Definition	Source
IMF	Intent Management Function		IG1251
AN	Autonomous Network		IG1253
AD	Autonomous Domain	TMF	IG1251
MAS	Multi-Agent Systems	Wikipedia	https://en.wikipedia.org/wiki/Multi-agent_system#:~:text=A%20multi%2Dagent%20system%20(MAS,a%20monolithic%20system%20to%20solve.
MDP	Markov Decision Processes	Article	https://ceur-ws.org/Vol-1588/paper1.pdf
CFS	Customer Facing Service	TMF	TMF071
RFS	Resource Facing Service	TMF	TMF071
SLA	Service Level Agreement	TMF	TMF071 GB922
KG	Knowledge Graph	Wikipedia	https://en.wikipedia.org/wiki/Knowledge_graph
AI	Artificial Intelligence	TMF	TMF071 GB1190 GB1003

© TM Forum 2025. All Rights Reserved.

17. References

Reference	Description	Source	Brief Use Summary
1	Intent in Autonomous Networks	TMF IG1253	"Intent in Autonomous Networks", v1.3.0.
2	Autonomous Networks Reference Architecture	TMF IG1251	"Autonomous Networks Reference Architecture", v1.0.1
3	Evolving Service Management Toward Intent-Driven Autonomous Networks	Ericsson Technology Review	Evolving Service Management Toward Intent-Driven Autonomous Networks, #3 March 2024, Jörg Niemöller, Elisabeth Müller, Massimiliano Mag giari, Kamal Maghsoudlou
4	Product Catalog Management API	TMF620	"Product Catalog Management API"
5	Service Catalog Management API	TMF633	"Service Catalog Management API"
6	Product Offering Qualification Management API	TMF679	"Product Offering Qualification Management API"
7	AI Management API	TMF915	"AI Management API"
8	Intent Management API	TMF921	"Intent Management API"
9	LLMs4OL: Large Language Models for Ontology Learning	arXiv:2307.16648v2 [cs.AI] 2 Aug 2023	H. B. Giglou, et al., "LLMs4OL: Large Language Models for Ontology Learning", August 2023
10	ReACT: "Synergizing Reasoning and Acting in Language Models"	https://arxiv.org/pdf/2210.03629	Y. Shunyu et. al., "Synergizing Reasoning and Acting in Language Models", ICLR 2023
11	"Cognitive Architecture for Language Agents (CoALA)"	Publish in transaction of Machine Learning March 2024, https://arxiv.org/abs/2309.02427	T. R. Sumers, et al., "Cognitive Architecture for Language Agents (CoALA)"
12	"Autonomous Cognitive Entity (ACE) Framework"	https://medium.com/@dave-shap/autonomous-agents-are-	D. Shapiro, "Autonomous Cognitive Entity (ACE) Framework", Medium Sept 2023,

© TM Forum 2025. All Rights Reserved.

Reference	Description	Source	Brief Use Summary
		here-introducing-the-ace-framework-a180af15d57c	
13	"Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects"	https://arxiv.org/abs/2401.03428	C. Yuheng, et al., "Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects", January 2024,
14	"Multi-agent System: A Survey from a Machine Learning Perspective"	https://www.cs.cmu.edu/~mmv/papers/MASsurvey.pdf	"Multi-agent System: A Survey from a Machine Learning Perspective", Autonomous Robotics Vol 8 No 3, July 2000,
15	"An Evaluation Survey of Knowledge-Based Approaches in Telecommunication Applications"	https://doi.org/10.3390/telecom5010006, https://www.mdpi.com/2653744	G. P. Koudouridis, et al., "An Evaluation Survey of Knowledge-Based Approaches in Telecommunication Applications", Telecom 2024, 5, 98–121.
16	"Towards Large Reasoning Models"	https://arxiv.org/pdf/2501.09686	X. Fengli, et al., "Towards Large Reasoning Models", Jan 2025,
17	UiPath	https://www.uipath.com/ai/agentic-ai	UiPath
18	Intelligent Agent	https://en.wikipedia.org/wiki/Intelligent_agent	Wikipedia https://en.wikipedia.org/wiki/Intelligent_agent
19	"What are AI agents"	https://www.ibm.com/think/topics/ai-agents	"What are AI Agents", IBM July 2024.
20	Is it an Agent, or Just a Program?: A Taxonomy for Autonomous Agents	https://www.researchgate.net/publication/221457111_Is_it_an_Agent_or_Just_a_Program_A_Tax	Is it an Agent, or Just a Program?: A Taxonomy for Autonomous Agents

© TM Forum 2025. All Rights Reserved.

Reference	Description	Source	Brief Use Summary
		onomy_for_Autonomous_Agents	
21	"An Introduction to Multi-Agent Systems"	Book, An Introduction to Multi Agent System", 2009 Wiley	M. Wooldridge, "An Introduction to Multi-Agent Systems"
22	"Artificial Intelligence: A Modern Approach"	Book, "Artificial Intelligence: A Modern Approach" (4th Edition). Pearson, 2021.	S. Russell, et al.,. "Artificial Intelligence: A Modern Approach"
23		"A Roadmap of Agent Research and Development. Autonomous Agents and Multi-Agent Systems", 1998	N. R. Jennings, et al., "A Roadmap of Agent Research and Development. Autonomous Agents and Multi-Agent Systems"
24	On autonomous domain orchestration and intent-policy binding.	ETSI ZSM 002 V1.1.1	On autonomous domain orchestration and intent-policy binding.
25	Service Models Explained	RFC 8969	Service Models Explained (why device-specific automation is hard).
26	Aether/SD-RAN – examples where programmable infrastructure is enabling early AI-based automation.	ONF	Aether/SD-RAN – examples where programmable infrastructure is enabling early AI-based automation.
27	IETF gNMI Telemetry Specification	https://github.com/openconfig/reference/blob/master/rpc/gnmi/gnmi-specification.md	IETF gNMI Telemetry Specification

© TM Forum 2025. All Rights Reserved.

Reference	Description	Source	Brief Use Summary
28	Requirements for Streaming Telemetry in Networks	RFC 8641	Requirements for Streaming Telemetry in Networks
29	"Differentiates between service-level intent and operational enforcement abstractions"	RFC 9315	"Differentiates between service-level intent and operational enforcement abstractions".
30	"Layered topology models that imply coordination boundaries within and across domains".	RFC 8345	"Layered topology models that imply coordination boundaries within and across domains".
31	"Topology Management API"	TMF686	"Topology Management API"
32	Combining Ontologies and Markov Logic Networks for Statistical Relational Mobile Network Analysis"	https://ceur-ws.org/Vol-1588/paper1.pdf	Kasper Apajalahti1, et al, "Combining Ontologies and Markov Logic Networks for Statistical Relational Mobile Network Analysis", Feb 2016,
33	Framework for Network Anomaly	RFC9232	Framework for Network Anomaly
34	Intent Classification and Taxonomy	RFC 9316	Intent Classification and Taxonomy
35	Intent-Based Policies in the RAN	ETSI TS 104 036 V12.0.0	Intent-Based Policies in the RAN
36	“Intent Common Model”	TMF TR290	“Intent Common Model”, v3.0.0
37	“Intent Common Model - Intent Expression”	TMF TR290A	“Intent Common Model - Intent Expression”, v.3.0.0
38	“Intent Common Model - Intent Reporting”	TMF TR290B	“Intent Common Model - Intent Reporting”, v.3.0.0
39	“Intent Common Model - Vocabulary Reference”	TMF TR290V	“Intent Common Model - Vocabulary Reference”, v3.0.0

© TM Forum 2025. All Rights Reserved.

Reference	Description	Source	Brief Use Summary
40	Future Military Intelligence CONOPS and S&T Investment Roadmap 2035-2050, Feb 2019.	https://nsiteam.com/social/wp-content/uploads/2019/04/Future-MI-CONOPS-and-ST-Roadmap-2035-2050_2-20-2019_FINAL.pdf	Haugland, Edward L. Future Military Intelligence CONOPS and S&T Investment Roadmap 2035-2050, Feb 2019.
41	“Intent Life Cycle Management and Interface”	TMF IG1253C	“Intent Life Cycle Management and Interface”, v1.3.0

© TM Forum 2025. All Rights Reserved.

18. Administrative Appendix

18.1. Document History

18.1.1. Version History

Version Number	Date Modified	Modified by:	Description of changes
1.0.0	11 Mar 2025	Rephael Benhamo	Initial Version, Author and Editor
1.0.0	19 May 2025	Rosie Wilson	Administrative Updated Prior to Publication
1.1.0	15 July 2025	Rephael Benhamo	Updated version
1.1.0	02 July 2025	Rosie Wilson	Administrative Updates
1.1.0	14 Aug 2025	Rephael Benhamo	Updated

18.1.2. Release History

Release Status	Date Modified	Modified by:	Description of changes
Pre-Production	19 May 2025	Rosie Wilson	Initial Publication
Pre-production	27-Oct-2025	Adrienne Walcott	Updated to Member Evaluated status

18.2. Acknowledgments

This document was prepared by the members of the TM Forum ODA Production team.

Team Member (@mention)	Company	Role*
Rephael Benhamo	Barnet Communications	Author and editor
Dave Milham	TM Forum	Co-editor/reviewer
Brad Peters	NBN Co	Contributor
Dymtro Gassanov	TM Forum	Reviewer
Peter Skoularikos	Telekinetics	Reviewer
Kevin McDonnell	Huawei	Additional Input
Mohit Goel	Telstra	Reviewer

**Select from: Project Chair, Project Co-Chair, Author, Editor, Key Contributor, Additional Input, Reviewer*

© TM Forum 2025. All Rights Reserved.